# Functional Bootstrapping for a Single LWE Ciphertext with $\tilde{O}(1)$ Polynomial Multiplications

Xiaopeng Zheng[1⋆], Hongbo Li[2,3], and Dingkang Wang[2,3]

[1] College of Mathematics and Computer Science, Shantou University, Shantou 515821, China
xiaopengzheng@stu.edu.cn
[2] State Key Laboratory of Mathematical Sciences, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China
{hli,dwang}@mmrc.iss.ac.cn
[3] School of Mathematical Sciences, University of Chinese Academy of Sciences, Beijing 100049, China

**Abstract.** Bootstrapping is the key technique that turns leveled homomorphic encryption into fully homomorphic encryption, but it remains a major efficiency bottleneck. Recent work by Z. Liu and Y. Wang (ASIACRYPT 2023) showed how to bootstrap $N$ LWE ciphertexts with total cost of $\widetilde{O}(N)$ polynomial multiplications based on the BFV scheme. However, their results achieve $\widetilde{O}(1)$ complexity only through amortization over large batches, and do not give a genuine non-amortized $\widetilde{O}(1)$ bound for a single ciphertext.

In this paper, we present a BFV-based functional bootstrapping algorithm for arbitrary functions over large plaintext spaces with total cost of $\widetilde{O}(1)$ polynomial multiplications for one LWE ciphertext. The same construction also supports small and moderate batches, and processes a batch of $m$ ciphertexts with total cost $\widetilde{O}(m)$ in the supported parameter range. The main technical ingredient is a sparse-packing polynomial-evaluation method for BFV ciphertexts, which exploits the duplicated-slot structure to evaluate an arbitrary polynomial on $m$ encrypted inputs with total cost of $\widetilde{O}(m)$.

We implement the scheme in Lattigo using the BFV scheme. At 128 bit security and on a single thread, bootstrapping an arbitrary function takes 3.15 seconds for one ciphertext encrypting a 9-bit plaintext and 3.77 seconds for 128 such ciphertexts in one batched invocation. For 16-bit plaintexts, it takes 10.63 seconds for one ciphertext and 18.07 seconds for 16 ciphertexts. These results show that non-amortized single-ciphertext functional bootstrapping, as well as small and moderate batch bootstrapping, can be practical for arbitrary functions over relatively large plaintext spaces.

**Keywords:** Homomorphic encryption · Homomorphic polynomial evaluation · Functional bootstrapping · Large plaintext spaces

⋆ Corresponding author.

## 1 Introduction

Fully homomorphic encryption (FHE) enables computation directly on encrypted data without requiring decryption, making it a central tool for privacy-preserving data processing, with applications such as private database queries and secure machine-learning inference [4,5,28,35]. Bootstrapping is the key operation that refreshes ciphertexts and thereby turns leveled homomorphic encryption into fully homomorphic encryption, but it remains one of the main efficiency bottlenecks in practice [23].

Since Gentry's Ph.D. thesis introduced the first fully homomorphic encryption scheme and the bootstrapping technique [23], a number of more efficient leveled homomorphic encryption schemes have been developed, including BGV [9], BFV [20], and CKKS [13]. A major advantage of this line of work is that it supports Single Instruction Multiple Data (SIMD) operations, allowing many plaintext values to be packed into a single ciphertext and processed in parallel, which makes it efficient for evaluating circuits with many inputs. Recent years have witnessed substantial progress in bootstrapping. For BGV and BFV, bootstrapping evolved from early boolean-circuit approaches [24] to SIMD-based improvements [25], and then to a more algebraic line of work [2,11,22,33], with a further optimization of the polynomial step given in [21]. For CKKS, Cheon et al. introduced bootstrapping for approximate homomorphic encryption [12], and subsequent works further improved its efficiency and precision [7,10,41].

Another influential direction is the FHEW/TFHE line, which is rooted in the GSW approximate-eigenvector framework [27]. Ducas and Micciancio [19] introduced FHEW and showed that LWE ciphertexts can be bootstrapped in less than a second by using a ring-GSW accumulator. Chillotti, Gama, Georgieva, and Izabachène [14] subsequently developed TFHE, where the GSW internal product is replaced by an external product between GSW-type and LWE-type ciphertexts, leading to substantially faster gate bootstrapping.

In the FHEW/TFHE line, recent work has gone beyond the negacyclic setting and has significantly expanded the supported function class and plaintext precision [38,43,56,57,49,29,46,58,18]. Here, functional bootstrapping refers to the stronger bootstrapping task that, given an LWE ciphertext encrypting $\mu \in \mathbb{Z}_p$ and a target function $f : \mathbb{Z}_p \to \mathbb{Z}_p$, outputs a fresh LWE ciphertext encrypting $f(\mu)$, with output noise small enough for correct decryption. In the batched case, the same operation is applied componentwise to many encrypted inputs. In parallel, batched and amortized bootstrapping has shown that many LWE ciphertexts can be processed together at very low amortized cost [50,17,44,45,31,47,30,3,1]. In particular, Liu and Wang [47] and Alexandru et al. [1] support arbitrary functional bootstrapping over relatively large plaintext spaces while achieving excellent amortized performance for very large batches.

Nevertheless, one important goal remains insufficiently addressed. Existing large-batch methods are designed to amortize their cost over thousands to tens of thousands of ciphertexts, and their guarantees do not directly translate into non-amortized $\tilde{O}(1)$-cost functional bootstrapping for a single LWE ciphertext. For example, in the 12-bit setting, [47] and [1] report total running times of about

1280 seconds and 599.9 seconds to bootstrap 32768 and 131072 ciphertexts, respectively. These designs are compelling when the workload is large enough to fully exploit amortization, but they may perform substantial unnecessary computation when the server needs to refresh only one ciphertext or only a modest number of ciphertexts.

This paper asks whether one can obtain functional bootstrapping for arbitrary functions over large plaintext spaces with total cost $\tilde{O}(1)$ for a single ciphertext, while still remaining efficient for small and moderate batch sizes. We answer this question affirmatively. We propose a large-plaintext functional bootstrapping scheme for arbitrary functions that uses only $\tilde{O}(1)$ polynomial multiplications for one LWE ciphertext. More generally, for $m$ ciphertexts, our method uses $\tilde{O}(m)$ polynomial multiplications and therefore retains amortized $\tilde{O}(1)$ complexity per ciphertext. We also implement the scheme and show that these asymptotic gains translate into practical performance.

*Motivating use cases for small and moderate batch sizes.* Small and moderate batches of scalar function evaluations arise naturally in privacy-preserving inference and exact table-lookup workloads. Examples include the following. (1) A clinical model for tabular data uses an MLP with hidden-layer widths 128, 64, and 32, with a ReLU applied after each hidden layer [59]. (2) In encrypted tumor classification, Hong et al. use a one-layer SNN classifier that outputs 11 class scores per patient record [34]. Processing an inference batch of $m$ patients therefore involves $11m$ scalar exponential evaluations in the softmax stage, which constitutes a small or moderate batch when only a small number of patients are classified at a time. (3) Autoregressive LLM decoding provides another example. At each decoding step with a context length of $n'$, each attention head produces $n'$ attention scores before softmax and hence requires $n'$ scalar exponential evaluations. Cachemir considers $n' = 512$ [60]. (4) Workloads involving exact table lookups also arise in AES transciphering [26]. Since a SubBytes layer applies the same 8-bit S-box independently to all 16 bytes of each block, processing 8 or 16 blocks in one such layer requires 128 or 256 S-box evaluations, respectively.

TFHE/FHEW-based programmable bootstrapping offers low latency for one or a few inputs, but higher-precision LUTs generally require more elaborate decomposition techniques. In contrast, fully packed BFV/CKKS methods cannot fully exploit their SIMD capacity at moderate batch size. Our method targets this middle regime by processing a moderate batch in one BFV ciphertext and using the remaining slots as computational resources. Accordingly, we target small and moderate batches over large plaintext space, with the goal of reducing the total latency of a single invocation rather than maximizing fully amortized throughput.

*Main technical contribution.* The main bottleneck in the large-plaintext functional bootstrapping methods of Liu and Wang [47] and Alexandru et al. [1] is homomorphic polynomial evaluation. In both works, this step is carried out using the Paterson–Stockmeyer method. For a polynomial with dense coefficients of degree $d$, it requires $d$ plaintext-ciphertext multiplications and $O(\sqrt{d})$

ciphertext-ciphertext multiplications. This cost is acceptable when amortized over very large batches, but it becomes a serious obstacle in the single-ciphertext and small-batch regimes.

Our main technical contribution is a new homomorphic polynomial evaluation algorithm for sparsely packed BFV ciphertexts. Suppose that a ciphertext sparsely encodes $m$ messages in a ring of dimension $N$. Under the assumptions $d = O(N)$ and $d < (N/m)^2$, our method evaluates a degree-$d$ polynomial on these $m$ messages using only $\tilde{O}(m)$ basic homomorphic operations. More precisely, it uses $\tilde{O}(1)$ ciphertext-ciphertext multiplications, $O(md/N)$ plaintext-ciphertext multiplications, and $\tilde{O}(\sqrt{md/N})$ rotations. In particular, when $m = 1$, this yields $\tilde{O}(1)$ complexity for homomorphic polynomial evaluation on a single encrypted input. This sparse-packing polynomial-evaluation algorithm is the key ingredient that enables our non-amortized $\tilde{O}(1)$ functional bootstrapping result.

### 1.1 Our contributions

**Homomorphic polynomial evaluation.** In this paper, we consider the BFV scheme. Let $R_q = (\mathbb{Z}_q)[X]/(X^N + 1)$ denote the plaintext space, where $q$ is the plaintext modulus and $N$ is the ring dimension, assumed to be a power of two. Suppose that $q$ is prime and satisfies $q \equiv 1 \pmod{2N}$. Then $X^N + 1$ splits completely over $\mathbb{Z}_q$, and hence $R_q \cong (\mathbb{Z}_q)^N$. Encoding a message as a vector in $(\mathbb{Z}_q)^N$ is called *slot encoding*.

For security considerations, the practical implementations of the BFV scheme typically use a large ring dimension, usually with $N \geq 4096$ [6]. On the other hand, in many applications, the message length is much smaller than the available slot capacity. In such cases, a natural approach is to replicate the message vector so as to fill all $N$ slots. More precisely, if the original message vector has length $m$, then it is repeated $N/m$ times and encoded as

$$\underbrace{(x_0, \ldots, x_{m-1}, x_0, \ldots, x_{m-1}, \ldots, x_0, \ldots, x_{m-1})}_{N/m \text{ times}}.$$

We refer to this representation as *sparse packing*, since it corresponds to a polynomial in $R_q$ with sparse coefficients. A similar sparse-packing strategy also appears in the CKKS bootstrapping literature [12].

Let $r = N/m$ denote the *repetition multiplicity*, namely, the number of repetitions in the sparse-packing representation. We study the following problem.

*Problem 1 (Batched homomorphic polynomial evaluation for sparsely packed ciphertexts).* Let $\mathsf{ct}$ be a BFV ciphertext encrypting $(x_0, x_1, \ldots, x_{m-1})$ with repetition multiplicity $r$, and let $f_0, \ldots, f_{m-1} \in \mathbb{Z}_q[x]$ be polynomials of degree $\leq d$. The goal is to efficiently compute a ciphertext encrypting

$$(f_0(x_0), f_1(x_1), \ldots, f_{m-1}(x_{m-1}))$$

with the same repetition multiplicity $r$.

In this paper, we propose efficient algorithms for batched homomorphic polynomial evaluation when $d+1 \le r^2$. In particular, we consider two cases, namely $d+1 \le r$ and $r < d+1 \le r^2$, and give an efficient algorithm for each case. For simplicity, we assume throughout that $N$, $m$, and $d+1$ are powers of two. Our main results are summarized as follows.

Under the conditions of Problem 1, one can homomorphically compute a ciphertext encrypting $(f_0(x_0), \ldots, f_{m-1}(x_{m-1}))$, still with repetition multiplicity $r$, with the complexity shown in Table 1.

**Table 1.** Complexity comparison of the proposed homomorphic polynomial evaluation algorithms and the Paterson–Stockmeyer baseline

| | | **Ours** | | **PS baseline** |
|---|---|---|---|---|
| **Condition** | $d+1 \le r$ | $r < d+1 < r^2$ | $d+1 = r^2$ | – |
| **CMult** | $\log(d+1)$ | $\log(d+1) + \frac{m(d+1)}{N}$ | $\log(d+1) + \frac{m(d+1)}{N}$ | $d$ |
| **Mult** | $2\log(d+1) - 2$ | $2\log(d+1) - 2$ | $2\log(d+1) - 2$ | $O(\sqrt{d})$ |
| **Rotation** | $\log(d+1)$ | $\log(\frac{N}{m}) + 2\sqrt{\frac{m(d+1)}{N}} - 2$ | $\log(\frac{N}{m}) + 4\sqrt{\frac{m(d+1)}{2N}} - 3$ | 0 |
| **Depth** | $\log(d+1) + 1$ | $\log(d+1) + 2$ | $\log(d+1) + 2$ | $\log(d+1)$ |

[a] **CMult**: number of plaintext-ciphertext multiplications.
[b] **Mult**: number of ciphertext-ciphertext multiplications.
[c] **Rotation**: number of homomorphic rotations.
[d] **Depth**: multiplicative depth.
[e] In this paper, all logarithms are base 2 unless otherwise stated.
[f] For the case $d+1 \le r$, it incorporates the optimization in Remark 1, which merges the coefficient multiplication into the power-generation step.

Therefore, assuming $d < (N/m)^2$ and $d = O(N)$, we have $d/N = O(1)$. Hence, our method evaluates $m$ polynomials using a total of $\tilde{O}(m)$ homomorphic operations, which gives amortized $\tilde{O}(1)$ complexity per polynomial.

Each homomorphic operation used in the circuit, namely ciphertext-ciphertext multiplication, plaintext-ciphertext multiplication, and homomorphic rotation, can be implemented using at most $\mathrm{poly}(\log Q)$ polynomial multiplications over $R_Q = \mathbb{Z}_Q[X]/(X^N+1)$. One polynomial multiplication means one multiplication of ring elements at modulus $Q$, which can be done with $O(N\log(N))\mathbb{Z}_Q$ operations using NTT. Hence, under the chosen leveled-BFV parameterization with $Q = \Theta(N^{\log N})$, each such operation costs at most $\tilde{O}(1)$ polynomial multiplications. Hence, in other words, our method evaluates $m$ polynomials using a total of $\tilde{O}(m)$ polynomial multiplications in $R_Q$, which gives amortized $\tilde{O}(1)$ complexity per polynomial.

**Large-Plaintext Functional Bootstrapping for Arbitrary Functions.** Building on the sparse polynomial-evaluation algorithm above, we obtain a functional bootstrapping scheme for arbitrary functions over large plaintext spaces. Assume throughout this subsection that $q$ is prime, $q \equiv 1 \pmod{2N}$, $q = O(N)$, $N = O(\lambda)$, and $n = O(\lambda)$, and that the underlying LWE and RLWE assump-

tions hold. Let $d$ be the smallest integer such that $d \geq q-1$ and $d+1$ is a power of two. The main result can be stated informally as follows.

**Theorem 1 (Informal).** *For every function $f : \mathbb{Z}_p \to \mathbb{Z}_p$, one can bootstrap a single LWE ciphertext encrypting $\mu \in \mathbb{Z}_p$ to a refreshed LWE ciphertext encrypting $f(\mu)$ using only $\tilde{O}(1)$ polynomial multiplications in $R_Q = \mathbb{Z}_Q[X]/(X^N+1)$. More generally, for every $m \leq N/\sqrt{d+1}$, one can bootstrap $m$ LWE ciphertexts for arbitrary functions using a total of $\tilde{O}(m)$ polynomial multiplications in $R_Q$.*

**Implementation.** We implement our general functional bootstrapping scheme in the homomorphic encryption library Lattigo [39] under a range of parameter settings and compare it with prior work. For each setting, we test two cases for the number $m$ of input LWE ciphertexts: $m = 1$ and the largest value of $m$ supported by that setting. We consider plaintext moduli corresponding to message precisions of 9, 12, 14, and 16 bits, and all parameter sets provide 128-bit security. The experimental results are reported in Table 2.

The experimental results show that our implementation supports bootstrapping over large plaintext spaces and achieves substantially smaller running time per invocation than existing batch bootstrapping methods. Our goal is not to improve the best amortized throughput in the very large-batch regime, but rather to reduce total latency in the single-ciphertext, small-batch, and moderate-batch regimes.

### 1.2 Related Works

**Homomorphic polynomial evaluation.** Our work is related to homomorphic polynomial evaluation. The classical Paterson–Stockmeyer method [53] evaluates a degree-$d$ polynomial with $O(\sqrt{d})$ nonscalar multiplications. Okada, Player, and Pohmann [51] exploited Galois structure to reduce the multiplicative cost for certain packed polynomial-evaluation settings and applied the method to BFV bootstrapping. Concurrent work by Zhao et al. [61] extends the norm- and trace-based methods of Okada et al. to non-power-of-two slot ranks. Wang and Wang [55] proposed an asymmetric BSGS algorithm for arbitrary dense polynomials in leveled homomorphic encryption, reducing the number of modulus and key switches. Park et al. [52] proposed a slim homomorphic polynomial evaluation algorithm for real-coefficient polynomials, based on a recursive sum-of-squares decomposition $P(x) = U(x)^2 + V(x)^2 - m$, which exploits sparsely packed CKKS ciphertexts and evaluates degree-$d$ polynomials using only $O(\log d)$ homomorphic operations.

**Functional bootstrapping and its extensions to larger precision in the TFHE line.** Programmable or functional bootstrapping is a central tool in the TFHE setting. Chillotti, Joye, and Paillier [15] showed how programmable bootstrapping can be used for efficient homomorphic inference of deep neural networks. Chillotti et al. [16] then improved programmable bootstrapping in

**Table 2.** Comparison for functional batch bootstrapping for various input bit-sizes

| Scheme(s) | Number of input messages | Number of input/output bits | Total time | Amortized time |
|---|---|---|---|---|
| [31] | 1024 | 8 | 871.8s | 0.851s |
| [30] | 8192 | 8 | 232s | 0.0283s |
| [47] | 32768 | 9 | 220s | 0.0067s |
| | 32768 | 12 | 1280s | 0.0391s |
| [3] | 4096 | 8 | 15.4s | 0.0038s |
| | 4096 | 10 | 50.3s | 0.0123s |
| [1] | 65536 | 9 | 63.96s | 0.0010s |
| | 131072 | 12 | 599.9s | 0.0046s |
| | 131072 | 14 | 2130.1s | 0.0163s |
| This work | 1 | 9 | 3.15s | 3.15s |
| | 128 | 9 | 3.77s | 0.0295s |
| | 1 | 12 | 8.36s | 8.36s |
| | 64 | 12 | 13.94s | 0.2178s |
| | 1 | 14 | 9.35s | 9.35s |
| | 32 | 14 | 16.88s | 0.5274s |
| | 1 | 16 | 10.63s | 10.63s |
| | 16 | 16 | 18.07s | 1.1295s |

* The results for prior works are taken directly from the corresponding papers, while the results for this work are obtained from our implementation. These comparisons are cross-implementation and cross-platform, and are intended to illustrate the regime in which our method is advantageous, rather than provide a cycle-accurate apples-to-apples benchmark.

TFHE, supporting larger precision and more efficient arithmetic circuits. Liu, Micciancio, and Polyakov [46] further studied large-precision sign evaluation in the FHEW/TFHE setting. Later works extended functional bootstrapping beyond the basic small-domain setting in several closely related directions. Kluczniak and Schild [38] initiated the full-domain line by removing the negacyclic restriction and supporting arbitrary lookup tables. Li et al. [43], Xia, Liu, and Wang [57], and Ma et al. [49] further improved full-domain and general-function bootstrapping, with Ma et al. [49] also improving homomorphic digit decomposition. Wang et al. [56] studied full-domain functional bootstrapping in the prime-cyclotomic setting. Xu et al. [58] and Duan et al. [18] developed large-plaintext variants with improved parameter efficiency and smaller bootstrapping keys. Overall, these works substantially broaden the supported function class and plaintext space, but they do not focus on the small-batch SIMD amortization setting considered in this work.

**Batched and amortized bootstrapping.** Batched and amortized bootstrapping have also been studied extensively. Micciancio and Sorrell [50] introduced

amortized bootstrapping for FHEW-like ciphertexts via ring packing. De Micheli, Kim, Micciancio, and Suhl [17] further improved this direction using ring automorphisms. Liu and Wang [44] introduced a new framework for SIMD bootstrapping in polynomial modulus and achieved amortized $\widetilde{O}(\lambda^{0.75})$ FHE multiplications and their sequel [45] improved this to amortized $\widetilde{O}(1)$ FHE multiplications. Guimarães, Pereira, and van Leeuwen [31] later revisited this line with a simpler and implemented construction. More recently, Guimarães and Pereira [30] proposed a fast amortized bootstrapping construction with small keys and polynomial noise overhead. For functional bootstrapping, Liu and Wang [47] showed that batched arbitrary functional bootstrapping can be achieved with amortized $\widetilde{O}(1)$ polynomial multiplications, while Lee, Min, and Song [40] proposed functional bootstrapping for packed ciphertexts via homomorphic LUT evaluation, supporting both FV-to-FV and CKKS-to-FV settings. These works greatly reduce the amortized cost, but they are mainly designed for large batch sizes. Among them, the closest to our work is [47]. Our construction follows the same general direction, but replaces the dense polynomial-evaluation step by a sparse-packing algorithm that is better suited to small and moderate batch sizes.

**Recent CKKS-based directions.** Recent works also showed that CKKS can support richer discrete computations than standard approximate arithmetic. Alexandru, Kim, and Polyakov [1] proposed general functional bootstrapping using CKKS, based on trigonometric Hermite interpolation, and obtained very low amortized cost for large batches. Bae, Kim, Stehlé, and Suvanto [3] proposed CKKS bootstrapping for small integers and bits: their SI-BTS supports small-integer functional bootstrapping, while their BB-BTS reduces the amortized cost of bit bootstrapping.

To make the comparison more explicit, Table 3 summarizes the most relevant prior works in terms of total cost, amortized cost, supported functionality, and implementation status.

### 1.3 Our Techniques

**Homomorphic polynomial evaluation.** We propose efficient algorithms for batched homomorphic polynomial evaluation when $d+1 \le r^2$. For simplicity, we explain the main ideas in the plaintext setting. All the operations involved can be performed homomorphically in the BFV scheme. We consider two cases: $d+1 \le r$ and $r < d+1 \le r^2$. Our main techniques are illustrated by two simple examples with $m = 1$.

Let $N = 8$ and $m = 1$. Then the repetition multiplicity is $r = 8$, and a single message is encoded in $\mathbb{Z}_q^8$ as $\boldsymbol{x} = (x, x, x, x, x, x, x, x)$. We aim to evaluate $f(x) = \sum_{i=0}^{7} a_i x^i$. Let $\boldsymbol{f} = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7)$. Let $\boldsymbol{x}_0 = \boldsymbol{x}$. First, compute

$$\begin{aligned} \boldsymbol{x}_1 &= \boldsymbol{x}_0 \odot \boldsymbol{x}_0 = (x^2, x^2, x^2, x^2, x^2, x^2, x^2, x^2), \\ \boldsymbol{x}_2 &= \boldsymbol{x}_1 \odot \boldsymbol{x}_1 = (x^4, x^4, x^4, x^4, x^4, x^4, x^4, x^4). \end{aligned} \tag{1}$$

**Table 3.** Comparison with prior work on batched LWE ciphertext bootstrapping in terms of asymptotic homomorphic operation counts

| Scheme(s) | Total cost | Messages | Amortized cost | Arbitrary function | Implementation |
|---|---|---|---|---|---|
| Ours (single-ciphertext) | $\tilde{O}(1)$ | 1 | $\tilde{O}(1)$ | Yes | Yes |
| Ours (maximal supported batch) | $\tilde{O}(\sqrt{n})$ | $O(\sqrt{n})$ | $\tilde{O}(1)$ | Yes | Yes |
| [19,14,42] | $O(n)$ | 1 | $O(n)$ | No | No |
| [50] | $\tilde{O}(3^{1/\epsilon} \cdot n^{1+\epsilon})$ | $O(n)$ | $\tilde{O}(3^{1/\epsilon} \cdot n^{\epsilon})$ | No | No |
| [31,17] | $O(\epsilon \cdot n^{1+1/\epsilon})$ | $O(n)$ | $O(\epsilon \cdot n^{1/\epsilon})$ | Yes | Yes (by [31]) |
| [30] | $\tilde{O}(nh)$ | $O(n)$ | $\tilde{O}(h)$ | Yes | Yes |
| [45] | $\tilde{O}(n)$ | $O(n)$ | $\tilde{O}(1)$ | No | No |
| [47] | $\tilde{O}(n)$ | $O(n)$ | $\tilde{O}(1)$ | Yes | Yes |
| [3] | $O(n^{1/\ell}) + O(p)$ | $O(n)$ | $O(1)$ | Yes | Yes |
| [1] | $O(n^{1/\ell}) + O(p)$ | $O(n)$ | $O(1)$ | Yes | Yes |

[a] Suppose that $n = O(\lambda)$, where $n$ denotes the LWE ciphertext dimension and $\lambda$ is the security parameter. If RLWE ciphertexts are used in the scheme, we further assume that $N = O(\lambda)$ and $N = O(n)$, where $N$ denotes the RLWE ciphertext dimension.
[b] In our scheme, we assume that $q > 2N$ and $q = O(N)$, where $q$ denotes the modulus. This is the same assumption as in [47]. Under these assumptions, the condition $m \leq N/\sqrt{q}$ implies $m = O(\sqrt{n})$.
[c] In [45], it is assumed that $q = O(\sqrt{N})$.
[d] For [3] and [1], the total cost consists of two parts. The term $O(n^{1/\ell})$ comes from the CoeffToSlot and SlotToCoeff procedures, where $\ell$ denotes the number of layers in the FFT decomposition. The term $O(p)$ comes from homomorphic polynomial evaluation, where $p$ is the plaintext modulus of the input LWE ciphertext. In the amortized complexity, we assume $p = O(n)$.
[e] For [30], $h$ denotes the Hamming weight of the sparse LWE/RLWE secret. The displayed bound uses the gap bound $B = O(n/h)$ for the sparse-polynomial multiplication step.

where $\odot$ denotes the Hadamard product (componentwise multiplication). Let

$$C = \begin{pmatrix} 0\,1\,0\,1\,0\,1\,0\,1 \\ 0\,0\,1\,1\,0\,0\,1\,1 \\ 0\,0\,0\,0\,1\,1\,1\,1 \end{pmatrix} = \begin{pmatrix} \boldsymbol{c}_0 \\ \boldsymbol{c}_1 \\ \boldsymbol{c}_2 \end{pmatrix} \text{ and } D = \begin{pmatrix} 1\,0\,1\,0\,1\,0\,1\,0 \\ 1\,1\,0\,0\,1\,1\,0\,0 \\ 1\,1\,1\,1\,0\,0\,0\,0 \end{pmatrix} = \begin{pmatrix} \boldsymbol{d}_0 \\ \boldsymbol{d}_1 \\ \boldsymbol{d}_2 \end{pmatrix}.$$

where the $j$-th column of $C$ is the binary expansion of $j-1$ and $D$ is the entrywise complement of $C$. Let

$$\begin{aligned} \boldsymbol{y}_0 &= \boldsymbol{x}_0 \odot (\boldsymbol{c}_0 \odot \boldsymbol{f}) + \boldsymbol{d}_0 \odot \boldsymbol{f} = (a_0, a_1x, a_2, a_3x, a_4, a_5x, a_6, a_7x), \\ \boldsymbol{y}_1 &= \boldsymbol{x}_1 \odot \boldsymbol{c}_1 + \boldsymbol{d}_1 = (1, 1, x^2, x^2, 1, 1, x^2, x^2), \\ \boldsymbol{y}_2 &= \boldsymbol{x}_2 \odot \boldsymbol{c}_2 + \boldsymbol{d}_2 = (1, 1, 1, 1, x^4, x^4, x^4, x^4), \end{aligned} \tag{2}$$

Then we obtain

$$\boldsymbol{y} = \boldsymbol{y}_0 \odot \boldsymbol{y}_1 \odot \boldsymbol{y}_2 = (a_0, a_1x, a_2x^2, a_3x^3, a_4x^4, a_5x^5, a_6x^6, a_7x^7). \tag{3}$$

We use the standard rotate-and-add technique to sum up all slots and obtain

$$\boldsymbol{h} = (f(x), f(x), f(x), f(x), f(x), f(x), f(x), f(x)).$$

When $d+1 > r$, we can no longer pack all powers $1, x, \ldots, x^d$ into the $r$ slots as in the above example. Therefore, the above procedure must be modified. For

example, let $N = 8$, $m = 1$ and $d = 31$. Then $r = N/m = 8$. We have $d+1 \leq r^2$. Suppose that we want to evaluate $f(x) = \sum_{i=0}^{31} a_i x^i$. Similarly, we compute

$$\boldsymbol{x}_i = (x^{2^i}, \ldots, x^{2^i}), \text{ for } i = 1, \ldots, 4, \tag{4}$$

where $4 = \log(d+1) - 1$. Applying the same construction as in (2), and (3), with the coefficient vector $\boldsymbol{f}$ replaced by $(1,1,1,1,1,1,1,1)$, we obtain

$$\boldsymbol{p} = (1, x, x^2, x^3, x^4, x^5, x^6, x^7). \tag{5}$$

Consider a matrix

$$F = \begin{pmatrix} a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ a_8 & a_9 & a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{16} & a_{17} & a_{18} & a_{19} & a_{20} & a_{21} & a_{22} & a_{23} \\ a_{24} & a_{25} & a_{26} & a_{27} & a_{28} & a_{29} & a_{30} & a_{31} \end{pmatrix} = \begin{pmatrix} \boldsymbol{f}_0 \\ \boldsymbol{f}_1 \\ \boldsymbol{f}_2 \\ \boldsymbol{f}_3 \end{pmatrix}$$

Perform a linear transformation of $\boldsymbol{p}$ by $F$ and obtain

$$\boldsymbol{h}^{(0)} = (h_0, h_1, h_2, h_3, h_0, h_1, h_2, h_3), \text{ where } h_j = \langle \boldsymbol{f}_j, \boldsymbol{p} \rangle = \sum_{i=0}^{7} a_{8j+i} x^i.$$

Applying the same construction as in (2) and (3) requires the vectors $\boldsymbol{x}_3 = (x^8, \ldots, x^8)$ and $\boldsymbol{x}_4 = (x^{16}, \ldots, x^{16})$. We also adjust the columns of $C$ so that they correspond to the bit decompositions of 0,1,2,3,0,1,2,3, respectively. We obtain $\boldsymbol{g} = (1, x^8, x^{16}, x^{24}, 1, x^8, x^{16}, x^{24})$. Then $\boldsymbol{g} \odot \boldsymbol{h}^{(0)} = (g_1, g_2, g_3, g_4, g_1, g_2, g_3, g_4)$, where

$$g_j = \sum_{i=0}^{7} a_{8(j-1)+i} x^{i+8(j-1)}, \qquad 1 \leq j \leq 4.$$

Finally, using the standard rotate-and-add technique, we sum $g_1, g_2, g_3, g_4$ and obtain $\boldsymbol{h} = (f(x), f(x), f(x), f(x), f(x), f(x), f(x), f(x))$.

**Large-Plaintext Functional Bootstrapping for Arbitrary Functions.** Our construction follows the basic idea of [47], with several modifications. We illustrate the method by a toy example in the case $m = 1$.

Let $\boldsymbol{c} = (\boldsymbol{a}, b) \in \mathbb{Z}_{17}^5$ be an LWE ciphertext satisfying

$$b = \langle \boldsymbol{a}, \boldsymbol{s} \rangle + \alpha\mu + e \pmod{q},$$

where $\mu \in \mathbb{Z}_2$ is the message, $\boldsymbol{s} \in \mathbb{Z}^4$ is the secret key, $\alpha = \lfloor 17/2 \rfloor = 8$, and $|e| < \lfloor 17/(2 \cdot 2) \rfloor = 4$. Suppose that we use a BFV scheme with ring dimension $N = 8$ and plaintext modulus $q = 17$, which is equal to the ciphertext modulus of LWE ciphertext. The corresponding bootstrapping key is a BFV ciphertext $\mathsf{ct}$ encrypting $(s_0, s_0, s_1, s_1, s_2, s_2, s_3, s_3)$, where $(s_0, s_1, s_2, s_3) = \boldsymbol{s}$.

In the bootstrapping procedure, we first perform a plaintext-ciphertext multiplication between $\mathsf{ct}$ and the plaintext vector $-(a_0, a_0, a_1, a_1, a_2, a_2, a_3, a_3)$, and

obtain a ciphertext encrypting $-(a_0s_0, a_0s_0, a_1s_1, a_1s_1, a_2s_2, a_2s_2, a_3s_3, a_3s_3)$. By the standard rotate-and-add technique, we sum the terms $a_0s_0$, $a_1s_1$, $a_2s_2$, $a_3s_3$ and obtain a ciphertext encrypting

$$-(\langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle, \langle \boldsymbol{a}, \boldsymbol{s}\rangle).$$

After adding the plaintext vector $(b, b, \ldots, b)$, we obtain a ciphertext $\mathsf{ct}_1$ encrypting $(\alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e)$.

Now let $g : \mathbb{Z}_2 \to \mathbb{Z}_2$ be an arbitrary function. Since $|e| < 4$, there exists a polynomial $f$ of degree 16 such that $f(\alpha\mu + e) = \alpha\, g(\mu)$. The construction of $f$ is given in [47], and we will briefly review it in Section 5. We then apply our homomorphic polynomial evaluation algorithm to $\mathsf{ct}_1$ and obtain a ciphertext $\mathsf{ct}_2$ encrypting $(\alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu))$. In this special case, all slots of $\mathsf{ct}_2$ are equal, so $\mathsf{ct}_2$ already encrypts the constant polynomial $\alpha g(\mu)$, and no SlotToCoeff step is needed. Finally, by modulus switching and key switching, we convert this BFV ciphertext into an LWE ciphertext, using the same extraction technique as in [47]. By properly choosing the BFV parameters, we ensure that the resulting LWE ciphertext encrypts $g(\mu)$ with new noise $e'$ satisfying $|e'| < 4$.

### 1.4 Organization of the paper

The remainder of the paper is organized as follows. Section 2 reviews the preliminaries, and Section 3 introduces the basic batched algorithms. Section 4 presents homomorphic polynomial evaluation for sparsely packed ciphertexts when $d+1 \le r^2$. Section 5 applies these algorithms to functional bootstrapping. Section 6 reports the implementation and experimental results, and Section 7 concludes the paper.

## 2 Preliminaries

### 2.1 LWE ciphertexts

For an integer $q \ge 2$, write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. We recall the Learning With Errors (LWE) problem of Regev [54] in the form used in this paper. Let $n$ be the dimension, let $q$ be the ciphertext modulus, let $\chi_s$ be a secret distribution over $\mathbb{Z}_q^n$, and let $\chi_e$ be an error distribution over $\mathbb{Z}$. For $\mathbf{s} \leftarrow \chi_s$, an LWE sample is a pair $(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ with $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ uniform and

$$b = \langle \mathbf{a}, \mathbf{s}\rangle + e \pmod q, \qquad e \leftarrow \chi_e.$$

The decisional LWE problem is to distinguish such samples from uniform samples in $\mathbb{Z}_q^n \times \mathbb{Z}_q$. Throughout the paper, we assume that this problem is hard for the parameter sets used below.

Let $p \ge 2$ be the plaintext modulus and set $\Delta = \lfloor q/p \rfloor$. An LWE encryption of $m \in \mathbb{Z}_p$ under $\mathbf{s}$ is a pair $\mathsf{ct} = (\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{a}$ is uniform and

$$b = \langle \mathbf{a}, \mathbf{s}\rangle + \Delta m + e \pmod q.$$

Decryption computes $v = b - \langle \mathbf{a}, \mathbf{s} \rangle \pmod{q}$ and rounds $pv/q$ to the nearest element of $\mathbb{Z}_p$. Hence decryption is correct whenever the error is smaller than the rounding radius, for example when $|e| < \lfloor q/(2p) \rfloor$.

We also use two standard LWE procedures. Modulus switching maps a ciphertext from modulus $q$ to a target modulus $q'$ by componentwise scaling and rounding:

$$\mathrm{ModSwitch}_{q \to q'}(\mathsf{ct}) = \left\lfloor \frac{q'}{q} \mathsf{ct} \right\rceil \in \mathbb{Z}_{q'}^{n+1}.$$

If the input error is bounded by $\beta$, then the output error is bounded in the standard form by $(q'/q)\beta + \sigma_{\mathrm{ms}}$, where $\sigma_{\mathrm{ms}}$ is the rounding contribution. Key switching changes the secret key while preserving the plaintext and increasing the error by a controlled amount determined by the key-switching parameters. We use both operations as black boxes and refer to [19] for their standard analysis.

### 2.2 BFV leveled homomorphic encryption

We briefly recall the ring-BFV scheme [8,20]. Let $N$ be a power of two and set $R = \mathbb{Z}[X]/(X^N + 1)$ and $R_M = R/MR$. In this subsection, $q$ denotes the BFV plaintext modulus and $Q$ the ciphertext modulus, with $q < Q$. We assume that $q$ is prime and $q \equiv 1 \pmod{2N}$. Then $X^N + 1$ splits over $\mathbb{Z}_q$, so $R_q \cong \mathbb{Z}_q^N$. We encode and decode vectors in $\mathbb{Z}_q^N$ by the corresponding inverse NTT and NTT maps, which gives the usual SIMD representation [32].

Let $\Delta = \lfloor Q/q \rfloor$, let $s \leftarrow \chi_{\mathsf{key}} \subset R$, and let $\chi_{\mathsf{err}}$ be an error distribution over $R$. A symmetric BFV encryption of $m(X) \in R_q$ is a pair $\mathsf{ct} = (a, b) \in R_Q^2$, where $a \leftarrow R_Q$ is uniform and

$$b = as + \Delta m(X) + e \pmod{Q}, \qquad e \leftarrow \chi_{\mathsf{err}}.$$

Decryption computes

$$\mu(X) = \left\lfloor \frac{q}{Q} \big(b - as \bmod Q\big) \right\rceil \in R_q$$

and decodes $\mu(X)$ to recover the message vector. Correctness holds as long as the coefficient error is sufficiently smaller than $Q/(2q)$.

Our construction uses the following BFV operations as black boxes:

$\mathsf{Add}$, $\mathsf{CMult}$, $\mathsf{Mult}$, $\mathsf{ColumnRotation}_i$, $\mathsf{RowRotation}$, KeySwitch, $\mathrm{ModSwitch}_{Q \to Q'}$.

$\mathsf{Add}$, $\mathsf{CMult}$, and $\mathsf{Mult}$ implement componentwise addition, plaintext–ciphertext multiplication, and ciphertext–ciphertext multiplication on SIMD slots, respectively. The operation KeySwitch changes the secret key and is used in particular for relinearization and rotations, while $\mathrm{ModSwitch}_{Q \to Q'}$ changes the ciphertext modulus and controls noise growth.

It remains to specify the rotations used in the paper. For

$$\boldsymbol{m} = (a_0, \ldots, a_{N/2-1}, b_0, \ldots, b_{N/2-1}) \in \mathbb{Z}_q^N,$$

we use the standard $2 \times (N/2)$ hypercube. The column rotation $\mathsf{ColumnRotation}_i$ rotates both rows by $i$ positions, and the row rotation $\mathsf{RowRotation}$ swaps the two rows:

$$\rho_i(\boldsymbol{m}) = (a_i, a_{i+1}, \ldots, a_{i-1},\, b_i, b_{i+1}, \ldots, b_{i-1}),$$
$$\tau(\boldsymbol{m}) = (b_0, \ldots, b_{N/2-1},\, a_0, \ldots, a_{N/2-1}),$$

where indices are taken modulo $N/2$. Thus $\mathrm{Dec}(\mathsf{ColumnRotation}_i(\mathsf{ct})) = \rho_i(\boldsymbol{m})$ and $\mathrm{Dec}(\mathsf{RowRotation}(\mathsf{ct})) = \tau(\boldsymbol{m})$, up to the usual key-switching error. We refer to [20,32,36] for the standard implementation details.

### 2.3 Linear Transformation

A homomorphic linear transformation (HLT), introduced in [32], has become a standard primitive in homomorphic machine learning. In their work, Halevi and Shoup proposed a diagonal method for multiplying a length-$r$ vector $\boldsymbol{m}$ by an $r \times r$ matrix $A$.

Subsequently, PEGASUS [48] generalized homomorphic linear transformations to the rectangular setting. Given a matrix $A \in \mathbb{Z}_q^{\ell \times r}$, its method conceptually tiles $A$ into a square structure, then extracts the required diagonal vectors, and then applies a BSGS strategy to evaluate the transformation efficiently. The algorithm explicitly distinguishes between the tall case $\ell \geq r$ and the short case $\ell < r$. As the short case will play an important role in Section 3.1, we present the corresponding procedure in detail here.

For the short case $\ell < r$, let $A = (a_{i,k}) \in \mathbb{Z}_q^{\ell \times r}$ and $\boldsymbol{m} \in \mathbb{Z}_q^r$. Assume for simplicity that $\ell \mid r$. To apply the diagonal method, we first conceptually extend $A$ to an $r \times r$ square matrix $\widetilde{A}$ by repeating its rows cyclically, namely

$$\widetilde{A}_{i,k} = a_{(i \bmod \ell),\, k}, \qquad 0 \leq i, k < r.$$

For each $0 \leq j < \ell$, we then extract the $j$-th diagonal vector $\boldsymbol{u}_j$ of $\widetilde{A}$, that is,

$$\boldsymbol{u}_j = \big(a_{0,j},\, a_{1,j+1},\, \ldots,\, a_{\ell-1,j+\ell-1},\, a_{0,j+\ell},\, a_{1,j+\ell+1},\, \ldots,\, a_{\ell-1,j+r-1}\big) \in \mathbb{Z}_q^r,$$

where the second index is taken modulo $r$. For convenience, we shall refer to $\boldsymbol{u}_j$ as the $j$-th diagonal of $A$ throughout this paper. Using these diagonal vectors, we first form the intermediate vector

$$\mathbf{y} = \sum_{0 \leq j < \ell} \big(\boldsymbol{u}_j \odot \rho_j(\boldsymbol{m})\big). \tag{6}$$

Unlike the square case, $\mathbf{y}$ is not yet equal to $\boldsymbol{m}A^{\mathrm{T}}$. Instead, one can recover $\boldsymbol{m}A^{\mathrm{T}}$ from $\mathbf{y}$ by a binary rotation-and-sum procedure: set $\mathbf{y}^{(0)} = \mathbf{y}$, and for $0 \leq i < \gamma$, where $\gamma = \log(r/\ell)$, define $\mathbf{y}^{(i+1)} = \mathbf{y}^{(i)} + \rho_{2^i \ell}\big(\mathbf{y}^{(i)}\big)$. After $\gamma$ steps, the vector $\mathbf{y}^{(\gamma)}$ becomes an $\ell$-periodic repetition of $\boldsymbol{m}A^{\mathrm{T}}$, namely

$$\mathbf{y}^{(\gamma)} = (\boldsymbol{m}A^{\mathrm{T}} \,\|\, \cdots \,\|\, \boldsymbol{m}A^{\mathrm{T}}),$$

Moreover, the computation of (6) can be accelerated by the baby-step giant-step (BSGS) technique. More precisely, if $\ell = bg$, then the intermediate vector $\mathbf{y}$ can be computed using $b + g - 2$ rotations and $\ell$ Hadamard products. Consequently, together with the additional $\log(r/\ell)$ rotation-and-sum steps, the total number of rotations in the short case is $b + g - 2 + \log(r/\ell)$, while the number of plaintext-ciphertext multiplications remains $\ell$.

### 2.4 Notations

Throughout the paper, $n$ denotes the LWE dimension, $p$ the LWE plaintext modulus, $N$ the BFV ring dimension, and $Q$ the BFV ciphertext modulus. We use $m$ for the number of input ciphertexts or packed messages, $r = N/m$ for the repetition multiplicity, and $d$ for the polynomial degree bound. The symbol $q$ denotes both the LWE ciphertext modulus and the BFV plaintext modulus. Unless otherwise stated, all logarithms are base 2. We also assume throughout that $N$, $m$, $r$, and $d+1$ are powers of two, that $N = rm$, and that $q \equiv 1 \pmod{2N}$. Under this condition, the BFV plaintext ring admits the usual slot decomposition, and the hypercube rotations introduced in Section 2.2 are available.

To describe the hypercube rotations more clearly, we partition a vector $\boldsymbol{m} \in \mathbb{Z}_q^N$ into two halves, and then further divide each half into $r/2$ consecutive blocks of length $m$. Accordingly, we index the entries by two subscripts: the first subscript records the position inside each block, ranging from 0 to $m-1$, and the second subscript records the block index, ranging from 0 to $r/2-1$. With this notation, we write $\boldsymbol{m}$ in the following two-dimensional hypercube form:

$$\begin{aligned}\boldsymbol{m} = (\ & a_{0,0}\ a_{1,0} \cdots a_{m-1,0} \cdots a_{0,r/2-1}\ a_{1,r/2-1} \cdots a_{m-1,r/2-1} \\ & b_{0,0}\ b_{1,0} \cdots b_{m-1,0} \cdots b_{0,r/2-1}\ b_{1,r/2-1} \cdots b_{m-1,r/2-1}\ ).\end{aligned} \tag{7}$$

We represent a vector $\boldsymbol{m} \in \mathbb{Z}_q^N$ as an $m \times r$ matrix. More precisely, define the bijections

$$\mathrm{mat} : \mathbb{Z}_q^N \to \mathbb{Z}_q^{m\times r}, \qquad \mathrm{rvec} : \mathbb{Z}_q^{m\times r} \to \mathbb{Z}_q^N,$$

where mat arranges the entries of $\boldsymbol{m}$ column by column into an $m \times r$ matrix, and rvec stacks the columns of a matrix into a vector. For $0 \le i \le m-1$, let $\boldsymbol{a}_i = (a_{i,0}, a_{i,1}, \ldots, a_{i,\frac{r}{2}-1})$ and $\boldsymbol{b}_i = (b_{i,0}, b_{i,1}, \ldots, b_{i,\frac{r}{2}-1})$. Then

$$\mathrm{mat}(\boldsymbol{m}) = \begin{pmatrix} a_{0,0} & \cdots & a_{0,\frac{r}{2}-1} & b_{0,0} & \cdots & b_{0,\frac{r}{2}-1} \\ a_{1,0} & \cdots & a_{1,\frac{r}{2}-1} & b_{1,0} & \cdots & b_{1,\frac{r}{2}-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & \cdots & a_{m-1,\frac{r}{2}-1} & b_{m-1,0} & \cdots & b_{m-1,\frac{r}{2}-1} \end{pmatrix} = \begin{pmatrix} \boldsymbol{a}_0 & \boldsymbol{b}_0 \\ \boldsymbol{a}_1 & \boldsymbol{b}_1 \\ \vdots & \vdots \\ \boldsymbol{a}_{m-1} & \boldsymbol{b}_{m-1} \end{pmatrix},$$

where each row is viewed as the concatenation of two vectors of length $r/2$.

For a vector $\boldsymbol{a} = (a_0, \ldots, a_{\frac{r}{2}-1}) \in \mathbb{Z}_q^{r/2}$, define

$$\sigma_j(\boldsymbol{a}) = (a_j, a_{j+1}, \ldots, a_{\frac{r}{2}-1}, a_0, a_1, \ldots, a_{j-1}),$$

where the indices are taken modulo $r/2$. For $M = \mathrm{mat}(\boldsymbol{m})$, define

$$\sigma_j(M) = \begin{pmatrix} \sigma_j(\boldsymbol{a}_0) & \sigma_j(\boldsymbol{b}_0) \\ \sigma_j(\boldsymbol{a}_1) & \sigma_j(\boldsymbol{b}_1) \\ \vdots & \vdots \\ \sigma_j(\boldsymbol{a}_{m-1}) & \sigma_j(\boldsymbol{b}_{m-1}) \end{pmatrix}, \qquad \tau(M) = \begin{pmatrix} \boldsymbol{b}_0 & \boldsymbol{a}_0 \\ \boldsymbol{b}_1 & \boldsymbol{a}_1 \\ \vdots & \vdots \\ \boldsymbol{b}_{m-1} & \boldsymbol{a}_{m-1} \end{pmatrix}.$$

Then, for each $j = 0, 1, \ldots, \frac{r}{2} - 1$, we have

$$\mathrm{mat}(\rho_{jm}(\boldsymbol{m})) = \sigma_j(\mathrm{mat}(\boldsymbol{m})), \qquad \mathrm{mat}(\tau(\boldsymbol{m})) = \tau(\mathrm{mat}(\boldsymbol{m})).$$

Hence, $\boldsymbol{m}$ can be viewed as packing $2m$ vectors of length $r/2$, and the rotations $\rho_{jm}$ simultaneously cyclically shift all these vectors by $j$ positions. As a result, one can perform linear transformations on them in a batched manner.

To describe sparsely packed vectors more conveniently, we recall the notion of the Kronecker product. This notation provides a concise way to represent the repetition of a vector multiple times, and will be used throughout the paper.

Let $\boldsymbol{x}$ be a plaintext sparsely packing $(x_0, \ldots, x_{m-1}) \in \mathbb{Z}_q^m$ with repetition multiplicity $r$, namely,

$$\boldsymbol{x} = \underbrace{(x_0, \ldots, x_{m-1},\, x_0, \ldots, x_{m-1},\, \ldots,\, x_0, \ldots, x_{m-1})}_{r \text{ times}} = \mathbf{1}_r \otimes (x_0, x_1, \ldots, x_{m-1}). \tag{8}$$

Then

$$\mathrm{mat}(\boldsymbol{x}) = \begin{pmatrix} x_0 & \cdots & x_0 & x_0 & \cdots & x_0 \\ x_1 & \cdots & x_1 & x_1 & \cdots & x_1 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ x_{m-1} & \cdots & x_{m-1} & x_{m-1} & \cdots & x_{m-1} \end{pmatrix}, \tag{9}$$

where each row consists of $r$ copies of the same entry. Therefore, $\boldsymbol{x}$ may be viewed as packing $m$ row vectors, each of which has the form $(x_i, \ldots, x_i) \in \mathbb{Z}_q^r$.

Recall the notion of multiplicative depth used in this paper. The depth of a ciphertext is denoted by $\mathrm{Depth}(\mathsf{ct})$. We assign depth 0 to every input ciphertext, and also set $\mathrm{Depth}(\mathsf{pt}) = 0$ for plaintexts. Only multiplications increase the multiplicative depth, while additions and rotations do not.

Recall also that, in the BFV scheme, noise growth is controlled by modulus switching along a modulus chain. Accordingly, the initial ciphertext modulus is chosen as a product of several large moduli, and each multiplication consumes one level of this chain. Therefore, if an algorithm has multiplicative depth $\ell$, then the BFV ciphertext modulus must contain at least $\ell + 1$ moduli.

## 3 Basic algorithms

### 3.1 Batched linear transformation

Under the hypercube representation in Section 2.4, diagonal HLT naturally extends to the following two batched settings.

– Algorithm 1 simultaneously applies diagonal HLT to the $2m$ subvectors of length $r/2$;
– Algorithm 2 applies diagonal HLT to the $m$ subvectors of length $r$ by reducing this case to Algorithm 1.

As discussed in Section 2.4, a vector $\boldsymbol{m} \in \mathbb{Z}_q^N$ may be viewed as packing $2m$ vectors of length $r/2$, and each of these vectors admits internal cyclic rotations. Therefore, the PEGASUS homomorphic linear transformation recalled in Section 2.3 naturally extends to the batched setting, where one simultaneously applies matrix multiplication to these $2m$ vectors of length $r/2$. The resulting procedure is described in Algorithm 1.

**Algorithm 1** Batched Linear Transformation over Ciphertexts with $2m$ blocks

**Input:** A ciphertext $\mathsf{ct}$ encrypting $\boldsymbol{m} \in \mathbb{Z}_q^N$ described in (7), with $N = rm$, and matrices $\{A_i, B_i\}_{i=0}^{m-1} \subset \mathbb{Z}_q^{\ell \times r/2}$, where $\ell \le r/2$ and $\ell, r, m$ are powers of two.

**Output:** A ciphertext encrypting $\boldsymbol{y} \in \mathbb{Z}_q^N$ such that

$$\mathrm{mat}(\boldsymbol{y}) = \begin{pmatrix} \mathbf{1}_s \otimes \boldsymbol{a}_0 A_0^{\mathrm{T}} & \mathbf{1}_s \otimes \boldsymbol{b}_0 B_0^{\mathrm{T}} \\ \mathbf{1}_s \otimes \boldsymbol{a}_1 A_1^{\mathrm{T}} & \mathbf{1}_s \otimes \boldsymbol{b}_1 B_1^{\mathrm{T}} \\ \vdots & \vdots \\ \mathbf{1}_s \otimes \boldsymbol{a}_{m-1} A_{m-1}^{\mathrm{T}} & \mathbf{1}_s \otimes \boldsymbol{b}_{m-1} B_{m-1}^{\mathrm{T}} \end{pmatrix},$$

where $s = r/(2\ell)$.

**Initial:** Let $\gamma \leftarrow \log s$. For each $j = 0, \ldots, \ell - 1$, let $\boldsymbol{u}_{ij}$ and $\boldsymbol{v}_{ij}$ be the $j$-th diagonals of $A_i$ and $B_i$ described in Section 2.3, respectively, and define

$$\boldsymbol{u}_j \leftarrow \mathrm{rvec} \begin{pmatrix} \boldsymbol{u}_{0,j} & \boldsymbol{v}_{0,j} \\ \boldsymbol{u}_{1,j} & \boldsymbol{v}_{1,j} \\ \vdots & \vdots \\ \boldsymbol{u}_{m-1,j} & \boldsymbol{v}_{m-1,j} \end{pmatrix}.$$

1: $\mathsf{ct}^{(1)} \leftarrow \mathsf{CMult}(\mathsf{ct}, \boldsymbol{u}_0)$;
2: **for** $j = 1, \ldots, \ell - 1$ **do**
3: $\quad \mathsf{ct}^{(1)} \leftarrow \mathsf{Add}(\mathsf{ct}^{(1)}, \mathsf{CMult}(\mathsf{ColumnRotation}(\mathsf{ct}, jm), \boldsymbol{u}_j))$;
4: **end for**
5: $\mathsf{ct}^{(2)} \leftarrow \mathsf{ct}^{(1)}$
6: **for** $i = 0, \ldots, \gamma - 1$ **do**
7: $\quad \mathsf{ct}^{(2)} \leftarrow \mathsf{Add}(\mathsf{ct}^{(2)}, \mathsf{ColumnRotation}(\mathsf{ct}^{(2)}, 2^i \ell m))$;
8: **end for**
9: **return** $\mathsf{ct}^{(2)}$.

Algorithm 1 handles batched linear transformations on $2m$ blocks of width $r/2$. We use it as a subroutine to realize batched linear transformations on $m$ blocks of width $r$. Let

$$\boldsymbol{c}_i = (a_{i,0}, a_{i,1}, \ldots, a_{i,r/2-1}, b_{i,0}, b_{i,1}, \ldots, b_{i,r/2-1}) = (\boldsymbol{a}_i, \boldsymbol{b}_i) \in \mathbb{Z}_q^r,$$

for $i = 0, 1, \ldots, m-1$. Given a ciphertext $\mathsf{ct}$ encrypting $\boldsymbol{m} \in \mathbb{Z}^N$ as in (7), and matrices $U_0, \ldots, U_{m-1} \in \mathbb{Z}_q^{\ell \times r}$, our goal is to compute a ciphertext encrypting $\boldsymbol{y} \in \mathbb{Z}_q^N$ such that

$$\mathrm{mat}(\boldsymbol{y}) = \begin{pmatrix} \mathbf{1}_{r/\ell} \otimes \boldsymbol{c}_0 U_0^{\mathrm{T}} \\ \mathbf{1}_{r/\ell} \otimes \boldsymbol{c}_1 U_1^{\mathrm{T}} \\ \vdots \\ \mathbf{1}_{r/\ell} \otimes \boldsymbol{c}_{m-1} U_{m-1}^{\mathrm{T}} \end{pmatrix}. \tag{10}$$

Algorithm 2 reduces this task to Algorithm 1 by decomposing each $U_i$ according to the relation between $\ell$ and $r$. If $\ell < r$, write

$$U_i = (A_i, B_i), \qquad A_i, B_i \in \mathbb{Z}_q^{\ell \times r/2}.$$

A single call to Algorithm 1 with input $\mathsf{ct}$ and $\{A_i, B_i\}_{i=0}^{m-1}$ computes the two partial products $\mathbf{1}_{r/(2\ell)} \otimes \boldsymbol{a}_i A_i^{\mathrm{T}}$, and $\mathbf{1}_{r/(2\ell)} \otimes \boldsymbol{b}_i B_i^{\mathrm{T}}$, for all $i$. Since

$$\mathbf{1}_{r/(2\ell)} \otimes \boldsymbol{c}_i U_i^{\mathrm{T}} = \mathbf{1}_{r/(2\ell)} \otimes \boldsymbol{a}_i A_i^{\mathrm{T}} + \mathbf{1}_{r/(2\ell)} \otimes \boldsymbol{b}_i B_i^{\mathrm{T}},$$

the desired ciphertext is obtained by applying the RowRotation and then adding. When $\ell = r$, write

$$U_i = \begin{pmatrix} A_i & B_i \\ C_i & D_i \end{pmatrix}, \qquad A_i, B_i, C_i, D_i \in \mathbb{Z}_q^{r/2 \times r/2}, \qquad i = 0, \ldots, m-1.$$

Then $\boldsymbol{c}_i U_i^{\mathrm{T}} = \big(\boldsymbol{a}_i A_i^{\mathrm{T}} + \boldsymbol{b}_i B_i^{\mathrm{T}},\ \boldsymbol{a}_i C_i^{\mathrm{T}} + \boldsymbol{b}_i D_i^{\mathrm{T}}\big)$. Accordingly, we apply Algorithm 1 twice: first to $\mathsf{ct}$ with $\{A_i, D_i\}_{i=0}^{m-1}$, and then to $\mathsf{RowRotation}(\mathsf{ct}, 1)$ with $\{B_i, C_i\}_{i=0}^{m-1}$. Therefore, $\mathsf{Add}(\mathsf{ct}^{(1)}, \mathsf{ct}^{(2)})$ encrypts the desired vector. In this way, Algorithm 2 reduces the case of $m$ blocks of width $r$ to the case of $2m$ blocks of width $r/2$ handled by Algorithm 1.

*Complexity of Algorithm 2.* If $\ell < r$, then Algorithm 2 requires $\ell$ plaintext-ciphertext multiplications and $\ell + \log\big(r/(2\ell)\big)$ homomorphic rotations. Moreover, if $\ell = bg$, then by the baby-step giant-step strategy, the number of homomorphic rotations can be reduced to $b + g - 1 + \log\big(r/(2\ell)\big)$.

If $\ell = r$, then Algorithm 2 requires $r$ plaintext-ciphertext multiplications and $r - 1$ homomorphic rotations. Moreover, if $r/2 = bg$, then by the baby-step giant-step strategy, the number of homomorphic rotations can be reduced to $2b + 2g - 3$.

Moreover, in both cases, the multiplicative depth is 1.

### 3.2 Batched generating powers

Let $\mathsf{ct}$ be a ciphertext encrypting $\boldsymbol{x} = \mathbf{1}_r \otimes (x_0, x_1, \ldots, x_{m-1})$. As discussed in (9) , the vector $\boldsymbol{x}$ may be viewed as packing $m$ row vectors, each of the form $(x_i, \ldots, x_i) \in \mathbb{Z}_q^r$. Following the technique reviewed in Section 1.3, we now explain how to generate powers in a batched manner. Let $s \leq r$, and assume

**Algorithm 2** BatchLT($\mathsf{ct}$,$\{U_0, \ldots, U_{m-1}\}$)

**Input:** A ciphertext $\mathsf{ct}$ encrypting $\boldsymbol{m} \in \mathbb{Z}_q^N$ as in (7), with $N = rm$, and matrices $U_0, \ldots, U_{m-1} \in \mathbb{Z}_q^{\ell \times r}$, where $\ell \leq r$, and $\ell, r, m$ are powers of two.
**Output:** A ciphertext encrypting $\boldsymbol{y} \in \mathbb{Z}_q^N$ such that $\mathrm{mat}(\boldsymbol{y})$ is described as (10)

1: **if** $\ell < r$ **then**
2: Write $U_i = (A_i, B_i)$ with $A_i, B_i \in \mathbb{Z}_q^{\ell \times r/2}$ for $i = 0, \ldots, m-1$.
3: Compute $\mathsf{ct}^{(1)}$ by Algorithm 1 with input $\mathsf{ct}$ and $\{A_i, B_i\}_{i=0}^{m-1}$.
4: **return** $\mathsf{Add}(\mathsf{ct}^{(1)}, \mathsf{RowRotation}(\mathsf{ct}^{(1)}, 1))$.
5: **else**
6: Write

$$U_i = \begin{pmatrix} A_i & B_i \\ C_i & D_i \end{pmatrix}$$

with $A_i, B_i, C_i, D_i \in \mathbb{Z}_q^{r/2 \times r/2}$ for $i = 0, \ldots, m-1$.
7: Set $\mathsf{ct}' \leftarrow \mathsf{RowRotation}(\mathsf{ct})$.
8: Compute $\mathsf{ct}^{(1)}$ by Algorithm 1 with input $\mathsf{ct}$ and $\{A_i, D_i\}_{i=0}^{m-1}$.
9: Compute $\mathsf{ct}^{(2)}$ by Algorithm 1 with input $\mathsf{ct}'$ and $\{B_i, C_i\}_{i=0}^{m-1}$.
10: **return** $\mathsf{Add}(\mathsf{ct}^{(1)}, \mathsf{ct}^{(2)})$.
11: **end if**

that $s$ is a power of two. Our goal is to compute a ciphertext $\mathsf{ct}_{\mathrm{out}}$ encrypting a vector $\boldsymbol{y} \in \mathbb{Z}_q^N$ with

$$\mathrm{mat}(\boldsymbol{y}) = \begin{pmatrix} \mathbf{1}_{r/s} \otimes (1, x_0, x_0^2, \ldots, x_0^{s-1}) \\ \mathbf{1}_{r/s} \otimes (1, x_1, x_1^2, \ldots, x_1^{s-1}) \\ \vdots \\ \mathbf{1}_{r/s} \otimes (1, x_{m-1}, x_{m-1}^2, \ldots, x_{m-1}^{s-1}) \end{pmatrix}. \tag{11}$$

The concrete procedure is given in Algorithm 3.

*Complexity of Algorithm 3.* The plaintext-ciphertext multiplications occur in Line 6 and Line 9, the number of which is 1 and $\log(s) - 1$, respectively, so the total number of plaintext-ciphertext multiplications is $\log(s)$. The ciphertext-ciphertext multiplications occur in Line 4 and Line 10, the number of which are $\log(s)-1$ and $\log(s)-1$, respectively, so the total number of ciphertext-ciphertext multiplications is $2\log(s) - 2$.

From Line 4, we have $\mathrm{Depth}(\mathsf{ct}_i) = i$, for $i = 1, \ldots, \ell - 1$. Furthermore, $\mathrm{Depth}(\mathsf{ct}'_i) = i + 1$ for $i = 1, \ldots, \ell - 1$. Therefore, in the $i$-th loop in Line 10, $\mathrm{Depth}(\mathsf{ct}'') = i + 2$ for $i = 1, \ldots, \ell - 1$. Then the ciphertext $\mathsf{ct}''$ returned in Lines 13 and 15 has depth $\ell + 1$.

## 4 Algorithms for homomorphic polynomial evaluation

In this section, we will present an algorithm for Problem 1 when $d + 1 \leq r^2$.

**Algorithm 3** $\mathsf{EvalPower}(\mathsf{ct}, s, b)$

**Input:** A ciphertext $\mathsf{ct}$ encrypting $\mathbf{1}_r \otimes (x_0, \ldots, x_{m-1})$, an integer $s \leq r$, where $m, r, s$ are powers of two, and an integer $b \in \{0, 1\}$. (The auxiliary output for $b = 1$ is used later to generate giant step powers in the case $r < d + 1 \leq r^2$ in Section 4.2).

**Output:** A ciphertext encrypting $\boldsymbol{y} \in \mathbb{Z}_q^N$ such that $\mathrm{mat}(\boldsymbol{y})$ is given in (11). In addition, if $b = 1$, the algorithm also outputs a ciphertext encrypting $\mathbf{1}_r \otimes (x_0^{s/2}, \ldots, x_{m-1}^{s/2})$.

1: **Initialization:** Set $s = 2^\ell$ and $t = r/s$. Let $C \in \mathbb{Z}_q^{\ell \times s}$ be the matrix whose columns are the binary expansions of $0, 1, \ldots, s-1$, and let $D = \mathbf{1}_{\ell \times s} - C$. Denote the $i$-th rows of $C$ and $D$ by $\boldsymbol{c}_i$ and $\boldsymbol{d}_i$, respectively. Let $C_i^{\mathrm{ext}} \in \mathbb{Z}_q^{m \times r}$ be the matrix whose every row is $\mathbf{1}_t \otimes \boldsymbol{c}_i$. Let $D_i^{\mathrm{ext}} \in \mathbb{Z}_q^{m \times r}$ be the matrix whose every row is $\mathbf{1}_t \otimes \boldsymbol{d}_i$. $\boldsymbol{c}_i^{\mathrm{ext}} \leftarrow \mathrm{rvec}(C_i^{\mathrm{ext}})$. $\boldsymbol{d}_i^{\mathrm{ext}} \leftarrow \mathrm{rvec}(D_i^{\mathrm{ext}})$.
2: $\mathsf{ct}_0 \leftarrow \mathsf{ct}$.
3: **for** $i = 1, \ldots, \ell - 1$ **do**
4: $\quad \mathsf{ct}_i \leftarrow \mathsf{Mult}(\mathsf{ct}_{i-1}, \mathsf{ct}_{i-1})$.
5: **end for**
6: $\mathsf{ct}_0' \leftarrow \mathsf{Add}(\mathsf{CMult}(\mathsf{ct}_0, \boldsymbol{c}_0^{\mathrm{ext}}), \boldsymbol{d}_0^{\mathrm{ext}})$.
7: $\mathsf{ct}'' \leftarrow \mathsf{ct}_0'$.
8: **for** $i = 1, \ldots, \ell - 1$ **do**
9: $\quad \mathsf{ct}_i' \leftarrow \mathsf{Add}(\mathsf{CMult}(\mathsf{ct}_i, \boldsymbol{c}_i^{\mathrm{ext}}), \boldsymbol{d}_i^{\mathrm{ext}})$.
10: $\quad \mathsf{ct}'' \leftarrow \mathsf{Mult}(\mathsf{ct}'', \mathsf{ct}_i')$.
11: **end for**
12: **if** $b = 0$ **then**
13: $\quad$ **return** $\mathsf{ct}''$.
14: **else**
15: $\quad$ **return** $\mathsf{ct}''$, $\mathsf{ct}_{\ell-1}$.
16: **end if**

### 4.1 The Case $d + 1 \leq r$

As shown in Section 3.2 and Algorithm 3, when $d + 1 \leq r$, one can homomorphically generate a ciphertext encrypting all powers $1, x_i, x_i^2, \ldots, x_i^d$ for each $x_i$. Therefore, to evaluate the polynomials $(f_0, f_1, \ldots, f_{m-1})$, it remains to multiply these powers by the corresponding coefficients of $(f_0, \ldots, f_{m-1})$ and then combine the resulting monomials by the standard rotate-and-sum procedure. This yields a ciphertext encrypting the desired polynomial evaluations. The detailed procedure is given in Algorithm 4.

*Complexity of Algorithm 4.* The call to $\mathsf{EvalPower}(\mathsf{ct}, d + 1, 0)$ in Line 1 requires $2\log(d+1) - 2$ ciphertext-ciphertext multiplications, $\log(d+1)$ plaintext-ciphertext multiplications, and multiplicative depth $\log(d + 1) + 1$. Line 2 contributes one additional plaintext-ciphertext multiplication, so the ciphertext $\mathsf{ct}_0'$ has depth $\log(d + 1) + 2$. The remaining steps involve only additions and rotations, and therefore do not increase the multiplicative depth. Hence, Algorithm 4 uses $2\log(d+1) - 2$ ciphertext-ciphertext multiplications, $\log(d+1)+1$

plaintext-ciphertext multiplications, and $\log(d+1)$ homomorphic rotations, and its multiplicative depth is $\log(d+1)+2$.

*Remark 1.* As in Section 1.3, one may incorporate the coefficients of the polynomials $f_i$ directly into the matrices $C$ used in Algorithm 3. In this way, the plaintext-ciphertext multiplication in Line 2 can be merged into the power-generation step. Consequently, the multiplicative depth of Algorithm 4 becomes $\log(d+1)+1$, and the number of plaintext-ciphertext multiplications is reduced by one.

**Algorithm 4** Homomorphic polynomial evaluation for sparsely packed ciphertexts when $d+1 \le r$

**Input:** A ciphertext $\mathsf{ct}$ encrypting $\mathbf{1}_r \otimes (x_0, \ldots, x_{m-1})$, and polynomials $f_0, \ldots, f_{m-1} \in \mathbb{Z}_q[x]$ of degree at most $d < r$ with $f_i(x) = \sum_{j=0}^{d} a_{ij} x^j$;

**Output:** A ciphertext encrypting $\mathbf{1}_r \otimes (f_0(x_0), f_1(x_1), \ldots, f_{m-1}(x_{m-1}))$ with repetition multiplicity $r$.

**Initial:** Let $d+1 = 2^\ell$ and $\boldsymbol{a}_i = (a_{i0}, a_{i1}, \ldots, a_{i,d})$ for $i = 0, \ldots, m-1$. Let $\boldsymbol{f} \in \mathbb{Z}_q^N$ with

$$\mathrm{mat}(\boldsymbol{f}) = \begin{pmatrix} \mathbf{1}_{r/(d+1)} \otimes (a_{0,0}, a_{0,1}, a_{0,2}, \ldots, a_{0,d}) \\ \mathbf{1}_{r/(d+1)} \otimes (a_{1,0}, a_{1,1}, a_{1,2}, \ldots, a_{1,d}) \\ \vdots \\ \mathbf{1}_{r/(d+1)} \otimes (a_{m-1,0}, a_{m-1,1}, a_{m-1,2}, \ldots, a_{m-1,d}) \end{pmatrix}.$$

1: $\mathsf{ct}' \leftarrow \mathsf{EvalPower}(\mathsf{ct}, d+1, 0)$;
2: $\mathsf{ct}'_0 \leftarrow \mathsf{CMult}(\mathsf{ct}', \boldsymbol{f})$;
3: **for** $i = 1, \ldots, \ell-1$ **do**
4: $\quad \mathsf{ct}'_i \leftarrow \mathsf{Add}(\mathsf{ct}'_{i-1}, \mathsf{ColumnRotation}(\mathsf{ct}'_{i-1}, m2^{i-1}))$;
5: **end for**
6: **if** $d+1 = r$ **then**
7: $\quad \mathsf{ct}'_\ell \leftarrow \mathsf{Add}(\mathsf{ct}'_{\ell-1}, \mathsf{RowRotation}(\mathsf{ct}'_{\ell-1}))$;
8: **else**
9: $\quad \mathsf{ct}'_\ell \leftarrow \mathsf{Add}(\mathsf{ct}'_{\ell-1}, \mathsf{ColumnRotation}(\mathsf{ct}'_{\ell-1}, m2^{\ell-1}))$;
10: **end if**
11: **return** $\mathsf{ct}'_\ell$.

### 4.2 The case $r < d+1 \le r^2$

Suppose $r < d+1 \le r^2$ and let $t = (d+1)/r$. We assume that $d+1$, $r$, and $t$ are all powers of two. For each $k = 0, \ldots, m-1$, the Paterson–Stockmeyer method [53] decomposes $f_k(x)$ as

$$g_{k,i}(x) = a_{k,ir} + a_{k,ir+1}x + \cdots + a_{k,ir+r-1}x^{r-1}, \qquad i = 0, \ldots, t-1.$$

Then $f_k(x)$ can be written as $f_k(x) = g_{k,0}(x) + g_{k,1}(x)x^r + \cdots + g_{k,t-1}(x)x^{(t-1)r}$.

Based on Algorithm 3, our algorithm for the case $r < d+1 \le r^2$ consists of three steps.

*Step 1. Generation of the power vectors.* Using Algorithm 3, we first compute two ciphertexts $\mathsf{ct}_1$ and $\mathsf{ct}_2$, encrypting vectors $\boldsymbol{y}_1$ and $\boldsymbol{y}_2$, respectively, such that

$$\mathrm{mat}(\boldsymbol{y}_1) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{r-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{r-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{m-1} & x_{m-1}^2 & \cdots & x_{m-1}^{r-1} \end{pmatrix}, \ \mathrm{mat}(\boldsymbol{y}_2) = \begin{pmatrix} \mathbf{1}_{r/t} \otimes (1, x_0^r, x_0^{2r}, \cdots, x_0^{(t-1)r}) \\ \mathbf{1}_{r/t} \otimes (1, x_1^r, x_1^{2r}, \cdots x_1^{(t-1)r}) \\ \vdots \\ \mathbf{1}_{r/t} \otimes (1, x_{m-1}^r, x_{m-1}^{2r}, \cdots x_{m-1}^{(t-1)r}) \end{pmatrix}.$$

*Step 2. Evaluation of the block polynomials.* For each $k = 0, \ldots, m-1$, define the coefficient matrix

$$G_k = \begin{pmatrix} a_{k,0} & a_{k,1} & \cdots a_{k,r-1} \\ a_{k,r} & a_{k,r+1} & \cdots a_{k,2r-1} \\ \vdots & \vdots & \vdots \\ a_{k,(t-1)r} & a_{k,(t-1)r+1} & \cdots a_{k,tr-1} \end{pmatrix}. \tag{12}$$

Then, by applying the batch linear transformation algorithm, Algorithm 2 to $\mathsf{ct}_1$, we obtain a ciphertext $\mathsf{ct}_3$ encrypting a vector $\boldsymbol{z}$ such that

$$\mathrm{mat}(\boldsymbol{z}) = \begin{pmatrix} \mathbf{1}_{r/t} \otimes (g_{0,0}(x_0), g_{0,1}(x_0), \cdots, g_{0,t-1}(x_0)) \\ \mathbf{1}_{r/t} \otimes (g_{1,0}(x_1), g_{1,1}(x_1), \cdots, g_{1,t-1}(x_1)) \\ \vdots \\ \mathbf{1}_{r/t} \otimes (g_{m-1,0}(x_{m-1}), g_{m-1,1}(x_{m-1}), \cdots, g_{m-1,t-1}(x_{m-1})) \end{pmatrix}.$$

*Step 3. Recombination.* Next, we multiply $\mathsf{ct}_3$ and $\mathsf{ct}_2$. Finally, applying the rotate-and-sum procedure within each row yields a ciphertext encrypting

$$\mathbf{1}_r \otimes \big(f_0(x_0),\, f_1(x_1),\, \ldots,\, f_{m-1}(x_{m-1})\big).$$

We summarize the above construction for the case $r < d+1 \leq r^2$ in Algorithm 5.

*Complexity of Algorithm 5.* Recall $t = (d+1)/r = m(d+1)/N$. The two calls to EvalPower in Lines 1 and 2 require in total $2\log(d+1) - 4$ ciphertext-ciphertext multiplications and $\log(d+1)$ plaintext-ciphertext multiplications. The number of plaintext-ciphertext multiplications in Line 4 is $t$. The multiplication in Line 2 and Line 5 contributes two more ciphertext-ciphertext multiplications, while the remaining operations are only additions and rotations. Therefore, the total number of ciphertext-ciphertext multiplications is $2\log(d+1) - 2$, the total number of plaintext-ciphertext multiplications is $t + \log(d+1)$, and the multiplicative depth is $\log(d+1) + 2$.

For the homomorphic rotations, if $d+1 < r^2$, then the algorithm uses $t + \log r - 1$ rotations. Moreover, if $t = bg$, then by applying the baby-step giant-step strategy to Algorithm 2 in Line 4, this number can be reduced to $b + g - 2 + \log r$. If $d+1 = r^2$, then the algorithm uses $r + \log r - 1$ rotations. Moreover, if $r/2 = bg$, then by applying the baby-step giant-step strategy to Algorithm 2 in Line 4, this number can be reduced to $2b + 2g + \log r - 3$.

**Algorithm 5** Homomorphic Polynomial evaluation for sparsely packed ciphertexts when $r < d+1 \leq r^2$

**Input:** A ciphertext $\mathsf{ct}$ encrypting $\boldsymbol{x} = \mathbf{1}_r \otimes (x_0, \ldots, x_{m-1})$, polynomials $f_0, \ldots, f_{m-1} \in \mathbb{Z}_q[x]$ of degree at most $d$, where $f_i(x) = \sum_{j=0}^{d} a_{ij} x^j$ and $r < d+1 \leq r^2$;
**Output:** A ciphertext encrypting $\mathbf{1}_r \otimes (f_0(x_0), f_1(x_1), \ldots, f_{m-1}(x_{m-1}))$.
**Initial:** Let $s = (d+1)/r$ and $\ell = \log(s)$.

$\mathsf{ct}_1, \mathsf{ct}' \leftarrow \mathsf{EvalPower}(\mathsf{ct}, r, 1)$;
$\mathsf{ct}' \leftarrow \mathsf{Mult}(\mathsf{ct}', \mathsf{ct}')$;
$\mathsf{ct}_2 \leftarrow \mathsf{EvalPower}(\mathsf{ct}', s, 0)$;
$\mathsf{ct}_3 \leftarrow \mathsf{BatchLT}(\mathsf{ct}_1, \{G_0, G_1, \ldots, G_{m-1}\})$; $G_k$ are defined in (12);
$\mathsf{ct}^{(0)} \leftarrow \mathsf{Mult}(\mathsf{ct}_2, \mathsf{ct}_3)$;
**for** $i = 1, \ldots, \ell - 1$ **do**
    $\mathsf{ct}^{(i)} \leftarrow \mathsf{Add}(\mathsf{ct}^{(i-1)}, \mathsf{ColumnRotation}(\mathsf{ct}^{(i-1)}, m2^{i-1}))$;
**end for**
**if** $d + 1 = r^2$ **then**
    $\mathsf{ct}^{(\ell)} \leftarrow \mathsf{Add}(\mathsf{ct}^{(\ell-1)}, \mathsf{RowRotation}(\mathsf{ct}^{(\ell-1)}))$;
**else**
    $\mathsf{ct}^{(\ell)} \leftarrow \mathsf{Add}(\mathsf{ct}^{(\ell-1)}, \mathsf{ColumnRotation}(\mathsf{ct}^{(\ell-1)}, m2^{\ell-1}))$;
**end if**
**return** $\mathsf{ct}^{(\ell)}$;

### 4.3 Detailed Comparison with Okada–Player–Pohmann [51]

To make the comparison precise, we consider the following common task: given a BFV ciphertext packing $m$ scalar messages $x_0, \ldots, x_{m-1}$ under a given slot layout and a public polynomial $f$, generate a single ciphertext packing $f(x_0)$, …, $f(x_{m-1})$ under the same slot layout.

We first instantiate the norm- and trace-based methods of Okada et al. [51] for this task. Let $N$ be the power-of-two ring dimension, let $m \mid N$, and set $r = N/m = 2^\ell$. Assume that there exists a prime $p$ satisfying $\mathrm{ord}_{2N}(p) = r$. Set $q := p^e$ and write $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}$. Then the BFV plaintext ring decomposes as

$$R/qR \cong \mathcal{S}^{N/r} = \mathcal{S}^m, \qquad \mathcal{S} = \mathbb{Z}_q[\zeta] \cong \mathbb{Z}_q[X]/(h),$$

where $h$ is monic and irreducible of degree $r$ over $\mathbb{F}_p$. Under this decomposition, the polynomial-evaluation algorithms of [51] can be applied componentwise to a ciphertext encrypting $(x_0, x_1, \ldots, x_{m-1}) \in \mathbb{Z}_q^m \subseteq \mathcal{S}^m$, and output a ciphertext encrypting $(f(x_0), f(x_1), \ldots, f(x_{m-1})) \in \mathbb{Z}_q^m \subseteq \mathcal{S}^m$. Here, each $x_i \in \mathbb{Z}_q$ is embedded into $\mathcal{S}$ as a constant polynomial.

In [51], two different methods are proposed, the norm-based and the trace-based. Let $x \in \mathbb{Z}_q \subseteq \mathcal{S}$ and $d = \deg(f)$. The first norm-based method handles the case $d < r$. Under the assumption that Heuristic 4.2 of [51] holds for a polynomial depending on $f$, [51] heuristically chooses a polynomial $g(X) = \sum_{i=0}^{\ell-1} c_i X^{2^i}$ such that $H(X) := X^r + f(X) + g(X)$ is irreducible modulo $p$, and then selects $\alpha \in \mathcal{S}$ such that $\mathrm{MiPo}_\alpha(X) = H(X)$, where $\mathrm{MiPo}_\alpha(X)$ denotes the monic minimal

polynomial of $\alpha$ over $\mathbb{Z}_q$. Since $\mathrm{Norm}_{\mathcal{S}/\mathbb{Z}_q}(\alpha - x) = \mathrm{MiPo}_\alpha(x)$, we have

$$f(x) = \mathrm{Norm}_{\mathcal{S}/\mathbb{Z}_q}(\alpha - x) - x^r - g(x). \tag{13}$$

Thus, $f(x)$ can be computed by homomorphically evaluating the right-hand side of (13). When $d = r + 1$, under Heuristic 4.4, [51] computes $f(x)$ homomorphically in a similar manner.

The trace-based method of [51] proceeds as follows. Let $f(X) = \sum_{i=0}^{d} f_i X^i$ with $d < r/2$. [51] uses $2\ell - 4$ ciphertext–ciphertext multiplications to compute

$$\beta = \sum_{i=0}^{r/2-1} x^i \zeta^i = \prod_{i=0}^{\ell-2} \left(1 + (x\zeta)^{2^i}\right). \tag{14}$$

Then $\beta$ is multiplied by $\sum_{i=0}^{d} f_i \zeta^{-i}$ and a trace-based coefficient-extraction procedure is applied to recover the constant coefficient of the product, which is exactly $f(x)$. In the special case $\mathrm{MiPo}(\zeta) = X^r + a$, the same construction can be extended to $d < r$.

Table 4 summarizes the comparison between the methods of [51] described above and our algorithms in Sections 4.1 and 4.2. The comparison highlights the following main differences.

**Table 4.** Comparison of homomorphic polynomial-evaluation methods for $m$ encrypted inputs. Here $r = N/m$ and $d$ denotes the target polynomial degree.

| | **This work** | **Okada–Player–Pohmann [51]** | |
|---|---|---|---|
| **Input layout** | $\mathbf{1}_r \otimes (x_0, \ldots, x_{m-1}) \in \mathbb{Z}_q^{mr}$ | $(x_0, \ldots, x_{m-1}) \in \mathbb{Z}_q^m \subseteq \mathcal{S}^m$, $\mathrm{rank}_{\mathbb{Z}_q}(\mathcal{S}) = r$ | |
| **Plaintext modulus** | $q$ prime and $\mathrm{ord}_{2N}(q) = 1$ | $q = p^e$, where $p$ is prime, $\mathrm{ord}_{2N}(p) = r$ | |
| **Main application** | LWE functional bootstrapping via BFV | Classical BFV bootstrapping, especially digit extraction | |
| **Methods** | Exploiting repeated slots | Norm-based | Trace-based |
| **Supported scope** | $d < r^2$ | $d < r$, Heuristic 4.2<br>$d = r + 1$, Heuristic 4.4 | $d < r/2$<br>$d < r$, $\mathrm{MiPo}(\zeta) = X^r + a$ |
| **Key-switch count** | $3\log(d+1) - 2$, $d < r$<br>$O\left(\sqrt{(d+1)/r} + \log r\right)$, $r \le d < r^2$ | $3\log r$, $d < r$<br>$2\log r + 1$, $d = r + 1$ | $3\log r - 4$, $d < r/2$<br>$3\log r - 2$, $r/2 \le d < r$ |
| **Multiplicative depth** | $\log(d+1) + 1$, $d < r$<br>$\log(d+1) + 2$, $r \le d < r^2$ | $\log r$, $d < r$<br>$\log r + 1$, $d = r + 1$ | $\log r - 1$, $d < r/2$<br>$\log r$, $r/2 \le d < r$ |

1. Algebraic-structure difference.
   (a) The plaintext structures are different. [51] uses $m$ rank-$r$ slots, with each message embedded as a constant, whereas our method uses a completely split ring of rank-one slots and repeats each of the $m$ messages $r$ times. Consequently, the two approaches also impose different requirements on the plaintext modulus.
   (b) The different plaintext structures also lead to different applications. Our method is tailored to the completely split rank-one-slot structure used in the BFV-based LWE functional-bootstrapping framework of [47],

whereas the implemented application of [51] is digit extraction in classical BFV bootstrapping with high-rank slots.

(c) The methods of [51] exploit the norm and trace within high-rank slots, whereas our method exploits repetition across rank-one slots.

2. Restriction condition difference.
   (a) The norm-based methods of [51] require heuristic assumptions on the target polynomial $f$, whereas extending the trace-based method from $d < r/2$ to $d < r$ requires $\mathrm{MiPo}(\zeta) = X^r + a$. Our method requires neither of these additional conditions.
   (b) The highest degree supported by [51] is $d = r + 1$, achieved only for certain monic polynomials under Heuristic 4.4. In contrast, our extended algorithm supports arbitrary polynomials satisfying $d < r^2$.
3. Theoretical complexity difference. For $d < r$, both approaches have logarithmic key-switching complexity and multiplicative depth. However, our cost depends on $d$, whereas the cost of [51] depends mainly on $r$. Thus, our method can have a lower cost when $d \ll r$.

*Remark 2 (Concurrent work).* The concurrent work [61] retains the high-rank plaintext-slot structure of [51] and extends both the norm- and trace-based approaches to non-power-of-two slot ranks. In the lower-degree range $d < r$, it extends the trace-based method to non-power-of-two slot ranks under the assumption that an irreducible binomial of degree $r$ exists. For $p > 2$ and $r < d \leq r \log p$, its higher-degree construction is primarily norm-based and, under a heuristic polynomial-decomposition assumption, uses $O(\sqrt{d/r} + \log d)$ ciphertext–ciphertext multiplications. Thus, it generalizes the high-rank-slot approach of [51], whereas our method exploits repetition across completely split rank-one slots and supports arbitrary polynomials with $d < r^2$ without a heuristic polynomial decomposition.

### 4.4 Potential extension to CKKS

Our repeated-slot polynomial-evaluation technique is not inherently restricted to BFV, since slotwise multiplications, plaintext masks, rotations, and homomorphic linear transformations are also available in CKKS. When the intermediate values are sparsely encoded in a repeated-slot layout, our method may accelerate polynomial evaluation in the monomial basis, such as the evaluation of a degree-$(p-1)$ polynomial in the CKKS functional bootstrapping of [1]. However, a complete CKKS instantiation requires scale management and a dedicated precision analysis. Since our power-vector construction does not directly apply to stages expressed in the Chebyshev basis, such as the EvalExp stage, we leave such an extension to future work.

## 5 Applications in Functional Bootstrapping

In this section, we apply the homomorphic polynomial evaluation algorithms developed in the previous section to the batched functional bootstrapping frame-

work of Liu and Wang [47]. Recall that functional bootstrapping refers to a procedure that, given an LWE ciphertext encrypting a message $m \in \mathbb{Z}_p$, outputs a refreshed LWE ciphertext encrypting $f(m)$ for a target function $f : \mathbb{Z}_p \to \mathbb{Z}_p$, while keeping the output noise within the correct decryption range. In particular, the algorithm of [47] takes as input a batch of LWE ciphertexts encrypting messages in $\mathbb{Z}_p$, and returns refreshed LWE ciphertexts encrypting the values of $f$ on the same messages.

At a high level, the construction of [47] consists of two main stages. In the first stage, it homomorphically computes $b - \langle \mathbf{a}, \mathbf{s} \rangle$ in the BFV scheme and then evaluates a polynomial $f_{\mathrm{poly}}$ representing the target lookup table. In the second stage, it converts the resulting BFV ciphertext back to LWE ciphertexts by applying SlotToCoeff, SampleExtract, key switching, and modulus switching.

We make a slight modification to this framework. We consider a batch size $m$ satisfying $m \leq \lceil N/\sqrt{q} \rceil$, and assume that $m$ is a power of two. Under this parameter choice, the first stage produces a sparsely packed BFV ciphertext rather than a fully packed one. We then replace the polynomial-evaluation step in [47] by our homomorphic polynomial evaluation algorithm for sparsely packed ciphertexts. For the SlotToCoeff step, we use a sparse version analogous to that of [12].

For clarity, we divide the modified construction into the following three steps.

**Step 1: Homomorphic decryption and sparse packing.** Let $r = N/m$. The first step is a sparse-packed version of the homomorphic decryption stage in the batched functional bootstrapping framework of [47]. It takes $m$ LWE ciphertexts $\mathsf{ct}_i = (\boldsymbol{a}_i, b_i) \in \mathbb{Z}_q^{n+1}$ with $b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + \alpha\mu_i + e_i \pmod q$, for $i = 0, \ldots, m-1$, and outputs a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \alpha\mu_1 + e_1, \ldots, \alpha\mu_{m-1} + e_{m-1}). \tag{15}$$

If $r \geq n$, let $\mathsf{ct}$ be a BFV ciphertext encrypting $\boldsymbol{s}^{(\mathrm{ext})}$, and let $\boldsymbol{a} \in \mathbb{Z}_q^N$, where

$$\mathrm{mat}(\boldsymbol{s}^{(\mathrm{ext})}) = \begin{pmatrix} \mathbf{1}_{r/n} \otimes \boldsymbol{s} \\ \vdots \\ \mathbf{1}_{r/n} \otimes \boldsymbol{s} \end{pmatrix}, \qquad \mathrm{mat}(\boldsymbol{a}) = \begin{pmatrix} \mathbf{1}_{r/n} \otimes \boldsymbol{a}_0 \\ \vdots \\ \mathbf{1}_{r/n} \otimes \boldsymbol{a}_{m-1} \end{pmatrix}.$$

Then $\mathsf{CMult}(\mathsf{ct}, -\boldsymbol{a})$ is a BFV ciphertext encrypting the slotwise products $-a_{ij}s_j$. After applying rotations and summing, and then adding $\boldsymbol{b}$, where the $i$-th row of $\mathrm{mat}(\boldsymbol{b})$ is $(b_i, b_i, \ldots, b_i) \in \mathbb{Z}_q^r$, we obtain a BFV ciphertext encrypting (15).

If $n > r$, let $t = n/r = mn/N$. Write

$$\boldsymbol{s} = (\boldsymbol{s}^{(0)}, \boldsymbol{s}^{(1)}, \ldots, \boldsymbol{s}^{(t-1)}), \qquad \boldsymbol{a}_i = (\boldsymbol{a}_i^{(0)}, \boldsymbol{a}_i^{(1)}, \ldots, \boldsymbol{a}_i^{(t-1)}),$$

where $\boldsymbol{s}^{(k)}, \boldsymbol{a}_i^{(k)} \in \mathbb{Z}_q^r$. For each $k = 0, \ldots, t-1$, let $\mathsf{ct}_k$ be a BFV ciphertext encrypting $\boldsymbol{s}_k^{(\mathrm{ext})}$, and let $\boldsymbol{a}^{(k)} \in \mathbb{Z}_q^N$ satisfy

$$\mathrm{mat}(\boldsymbol{s}_k^{(\mathrm{ext})}) = \begin{pmatrix} \boldsymbol{s}^{(k)} \\ \vdots \\ \boldsymbol{s}^{(k)} \end{pmatrix}, \qquad \mathrm{mat}(\boldsymbol{a}^{(k)}) = \begin{pmatrix} \boldsymbol{a}_0^{(k)} \\ \vdots \\ \boldsymbol{a}_{m-1}^{(k)} \end{pmatrix}.$$

For each $k = 0, \ldots, t-1$, $\mathsf{CMult}(\mathsf{ct}_k, -\boldsymbol{a}^{(k)})$ encrypts a vector whose $i$-th row is $-\boldsymbol{a}_i^{(k)} \odot \boldsymbol{s}^{(k)}$. Summing these ciphertexts over $k$ and then applying a single rotate-and-sum within each row yields a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes \big(-\langle \boldsymbol{a}_0, \boldsymbol{s}\rangle, \ \ldots, \ -\langle \boldsymbol{a}_{m-1}, \boldsymbol{s}\rangle\big).$$

Finally, adding $\boldsymbol{b}$, where the $i$-th row of $\mathrm{mat}(\boldsymbol{b})$ is $(b_i, b_i, \ldots, b_i) \in \mathbb{Z}_q^r$, gives a BFV ciphertext encrypting (15).

**Step 2. Homomorphic polynomial evaluation on sparse ciphertexts.** Following the construction of Liu and Wang [47], we work with BFV plaintext modulus $q$. Let $\alpha = \lfloor q/p \rfloor$ and $B = \lfloor (q/(2p)) \rfloor$. For each $\mu \in \mathbb{Z}_p$, define

$$I_\mu = \{\, \alpha\mu + e \pmod q : \ |e| < B \,\} \subseteq \mathbb{Z}_q.$$

Since $\alpha = \lfloor q/p \rfloor$ and $B = \lfloor q/(2p) \rfloor$, the sets $I_\mu$ are disjoint. We then define a lookup table $\mathrm{LUT} : \mathbb{Z}_q \to \mathbb{Z}_q$ by $\mathrm{LUT}(x) = \alpha\, f(\mu)$, for all $x \in I_\mu$, and assign zero to $\mathrm{LUT}(x)$ outside $\bigcup_{\mu \in \mathbb{Z}_p} I_\mu$. Since $q$ is prime, every function on $\mathbb{Z}_q$ is represented by a unique polynomial of degree at most $q-1$. In particular, we define

$$f_{\mathrm{poly}}(X) = \mathrm{LUT}(0) - \sum_{i=1}^{q-1} X^i \sum_{a=0}^{q-1} \mathrm{LUT}(a)\, a^{\,q-1-i}.$$

Then $f_{\mathrm{poly}}(x) = \mathrm{LUT}(x)$ for every $x \in \mathbb{Z}_q$. Therefore, for every $x = \alpha\mu + e \in \mathbb{Z}_q$ with $|e| < B$, we have $f_{\mathrm{poly}}(x) = \alpha f(\mu)$. Hence, homomorphically evaluating $f_{\mathrm{poly}}$ on a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \alpha\mu_1 + e_1, \ldots, \alpha\mu_{m-1} + e_{m-1}),$$

where $|e_i| < B$, obtains a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha f(\mu_0), \alpha f(\mu_1), \ldots, \alpha f(\mu_{m-1})). \tag{16}$$

**Step 3. Sparse SlotToCoeff and extraction back to LWE ciphertexts.** Let $N = 2tm$, so that $t = r/2$. After Step 2, the BFV plaintext has the repeated-slot form $\boldsymbol{x} = \mathbf{1}_r \otimes (x_0, x_1, \ldots, x_{m-1}) \in \mathbb{Z}_q^N, x_i = \alpha f(\mu_i)$. Up to a fixed public permutation of the slots, this vector is a $t$-sparse packing together with the

row-invariance condition $\tau(\boldsymbol{x}) = \boldsymbol{x}$. In the sparse-packing framework of [12], a $t$-sparse plaintext is supported on the subring

$$\mathbb{Z}_q[X^t]/(X^N+1) \cong \mathbb{Z}_q[Y]/(Y^{2m}+1), \qquad Y = X^t.$$

The additional condition $\tau(\boldsymbol{x}) = \boldsymbol{x}$ restricts this sparse subring to its $\tau$-invariant part. Equivalently, the relevant coefficient space is spanned by plaintexts of the form $a_0 + \sum_{j=1}^{m-1} a_j \big(X^{jt} - X^{(2m-j)t}\big)$. This is analogous to the conjugate-invariant encoding of [37].

We use the SlotToCoeff map restricted to this $m$-dimensional repeated-slot subspace. The ordinary restricted SlotToCoeff map and the map that sends $\mathbf{1}_r \otimes (u_0, \ldots, u_{m-1})$ to $u_0 + \sum_{j=1}^{m-1} u_j \big(X^{jt} - X^{(2m-j)t}\big)$ differ only by a public invertible $m \times m$ linear change of basis on the sparse slots. We absorb this public change of basis, together with the usual BFV encoding normalizations, into the plaintext diagonals of the homomorphic linear transform. Hence, after applying the sparse SlotToCoeff transform, the coefficient of $X^{tj}$ is $\alpha f(\mu_j)$, for $j = 0, \ldots, m-1$, while the conjugate positions $X^{(2m-j)t}$ carry the corresponding negative coefficients.

Since the restricted transform is linear and has only $m$ independent diagonals, it can be evaluated by the standard diagonal method with $m$ plaintext-ciphertext multiplications, and by the usual baby-step giant-step optimization with $O(\sqrt{m})$ rotations, as in the sparse SlotToCoeff procedures used in CKKS bootstrapping [12]. Finally, following Liu and Wang [47, Section 4.5], we perform BFV key switching and modulus switching before extraction. We then apply SampleExtract only to the coefficients of $X^{tj}$, for $j = 0, \ldots, m-1$, which yields $m$ LWE ciphertexts encrypting $f(\mu_0), \ldots, f(\mu_{m-1})$.

**Summary.** Combining the sparse homomorphic decryption in Step 1, the sparse polynomial evaluation in Step 2, and the sparse SlotToCoeff/extraction procedure in Step 3, we obtain the following formal guarantee for the resulting functional bootstrapping construction.

Let

$$\sigma_{\mathrm{res}} := \sqrt{\left(\frac{q}{Q'}(\sigma + \sigma_{\mathrm{ks}})\right)^2 + \sigma_{\mathrm{ms}}^2},$$

where $\sigma$ is the standard deviation of the coefficient noise after Step 2 and the sparse SlotToCoeff procedure but before the BFV key-switching step, $Q'$ is the BFV ciphertext modulus after the BFV key-switching step, $\sigma_{\mathrm{ks}}$ is the additional noise introduced by the BFV key switching, and $\sigma_{\mathrm{ms}}$ is the rounding noise introduced by the final LWE modulus-switching step.

**Theorem 2.** *Adopt the notation and standing assumptions of Section 2.4. Let $f : \mathbb{Z}_p \to \mathbb{Z}_p$ be any function, and let $m \le N/\sqrt{d+1}$. Suppose that $\mathsf{ct}_i = (\boldsymbol{a}_i, b_i) \in \mathbb{Z}_q^{n+1}$, for $i = 0, \ldots, m-1$, are LWE ciphertexts encrypting $\mu_i \in \mathbb{Z}_p$ under the same LWE secret key $\boldsymbol{s}$, with error magnitude less than $\lfloor q/(2p) \rfloor$. Assume moreover that the BFV parameters are chosen so that every intermediate*

*BFV ciphertext in Steps 1–3 remains decryptable. Then the construction in Section 5 outputs LWE ciphertexts* $\mathsf{ct}_i^{\mathrm{out}} = (\boldsymbol{a}_i^{\mathrm{out}}, b_i^{\mathrm{out}}) \in \mathbb{Z}_q^{n+1},\ i = 0, \ldots, m-1$, *encrypting* $f(\mu_i)$ *under the same secret key* $\boldsymbol{s}$*, each with output-noise standard deviation at most* $\sigma_{\mathrm{res}}$*, for* $i = 0, \ldots, m-1$.

*Proof (Proof sketch).* We prove the theorem by correctness and noise analysis.

*Correctness.* In Step 1, the sparse homomorphic decryption procedure outputs a BFV ciphertext encrypting $\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \ldots, \alpha\mu_{m-1} + e_{m-1})$. In Step 2, by the definition of $f_{\mathrm{poly}}$, we have $f_{\mathrm{poly}}(\alpha\mu_i + e_i) = \alpha f(\mu_i)$ whenever $|e_i| < \lfloor q/(2p) \rfloor$. Hence Step 2 outputs a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha f(\mu_0), \ldots, \alpha f(\mu_{m-1})).$$

Finally, Step 3 applies sparse SlotToCoeff, SampleExtract, key switching, and modulus switching, which preserve the underlying plaintext. Therefore the output LWE ciphertexts encrypt $f(\mu_0), \ldots, f(\mu_{m-1})$.

*Noise analysis.* Let $\sigma$ be the standard deviation of the coefficient noise after Step 2 and the sparse SlotToCoeff procedure. Since SampleExtract only selects coefficients and changes signs, it does not increase the noise standard deviation. The subsequent BFV key switching adds noise of standard deviation $\sigma_{\mathrm{ks}}$. After modulus switching from $Q'$ to $q$, the previous noise is scaled by $q/Q'$, and the final modulus-switching rounding contributes noise of standard deviation $\sigma_{\mathrm{ms}}$. Thus the output LWE noise has standard deviation $\sigma_{\mathrm{res}} = \sqrt{\left(\frac{q}{Q'}(\sigma + \sigma_{\mathrm{ks}})\right)^2 + \sigma_{\mathrm{ms}}^2}$. This gives the claimed output noise bound. □

*Complexity.* Step 1 requires $\log(\min\{r, n\})$ homomorphic rotations and $\lceil n/r \rceil$ plaintext-ciphertext multiplications. Step 2 requires $2\log(d+1) - 2$ ciphertext-ciphertext multiplications, $\log(d+1) + m(d+1)/N$ plaintext-ciphertext multiplications, and $\tilde{O}(\sqrt{m(d+1)/N})$ rotations. Step 3 requires $m$ plaintext-ciphertext multiplications and $O(\sqrt{m})$ rotations, together with the final key-switching and modulus-switching steps. Since $d+1$ is the smallest power of two satisfying $d \geq q - 1$ and $q = O(N)$, we have $d = O(N)$. Under the standing assumptions $N = O(\lambda)$ and $n = O(\lambda)$, the total number of BFV homomorphic operations is therefore $\tilde{O}(m)$. As each such operation can be implemented using $\tilde{O}(1)$ polynomial multiplications in $R_Q$, the total complexity is $\tilde{O}(m)$ polynomial multiplications in $R_Q$. In particular, when $m = 1$, the complexity is $\tilde{O}(1)$.

*Failure probability.* Assume that $Q'$ is chosen so that $((q/Q')(\sigma + \sigma_{\mathrm{ks}}))^2$ is negligible compared with the final modulus-switching noise. Following the random-fractional-part heuristic for non-randomized modulus switching of Ducas and Micciancio [19], the final rounding error is modeled as $e_{\mathrm{ms}} = \eta_b - \sum_{i=0}^{n-1} s_i \eta_i$, where $\eta_b, \eta_0, \ldots, \eta_{n-1}$ are independent centered uniform variables on $[-1/2, 1/2]$. For a dense sign LWE secret, we have $\|\boldsymbol{s}\|_2^2 = n$, and hence $\sigma_{\mathrm{res}}^2 \approx \sigma_{\mathrm{ms}}^2 \approx (n+1)/12$. Let $B = \lfloor q/(2p) \rfloor$ be the LWE decryption threshold. Under this subgaussian model, the decryption-failure probability of one output ciphertext is bounded by $p_{\mathrm{fail}}^{(1)} = \Pr[|e_{\mathrm{out}}| \geq B] \leq 2\exp\left(-\frac{6B^2}{n+1}\right)$.

## 6 Experiments

We implemented the functional bootstrapping algorithm described in Section 5 in Go on top of the Lattigo homomorphic encryption library (version 6.2.0), using its BFV interface, which in Lattigo v6 is implemented through the unified BFV/BGV integer-arithmetic backend. Our implementation supports arbitrary functions over the LWE plaintext space. In the following, we report experimental results for LWE plaintext moduli $p = 2^9, 2^{12}, 2^{14}, 2^{16}$. In all experiments, the LWE dimension is fixed to $n = 2048$, the BFV plaintext modulus $T$ is set equal to the LWE ciphertext modulus $q$, and the LWE encoding factor is $\alpha = \lfloor q/p \rfloor$.

The input LWE ciphertexts have the form $b = \langle a, s \rangle + \alpha\mu + e \pmod q$. Unless an explicit secret vector is supplied, the LWE secret $s$ is sampled coefficient-wise from the uniform ternary distribution on $\{-1, 0, 1\}$. The public LWE vector $a$ is sampled uniformly from $\mathbb{Z}_q^n$. The input LWE error $e$ is sampled as a rounded discrete Gaussian with standard deviation 3.2, rejection-sampled so that $|e| < \lfloor q/(2p) \rfloor$. For the underlying BFV/RLWE scheme, we do not override Lattigo's default secret and error distributions in the parameter literal. Thus the BFV/RLWE secret uses Lattigo's default ternary distribution, and the BFV/RLWE error uses Lattigo's default discrete Gaussian distribution, with standard deviation 3.2 and truncation bound 19.2. All parameter sets were chosen to provide at least 128-bit security, following the parameter selection guidelines in [6].

All experiments were conducted on a Windows desktop machine running Microsoft Windows 11 Home, version 10.0.22631, build 22631. The machine was equipped with an Intel(R) Core(TM) i9-14900K processor with 24 physical cores and 32 logical processors, and 31.69 GiB of RAM. The experiments were run from Windows PowerShell 5.1.22621.6133 using Go 1.26.3 for windows/amd64. Unless otherwise stated, all reported timings are online times for one invocation of the corresponding bootstrapping procedure, averaged over 100 independent runs. No application-level parallelization across independent bootstrapping instances was used.

The main comparison results are summarized in Table 2. The concrete parameter sets are listed in Table 5, and Table 6 gives the runtime breakdown, including the main online steps, key sizes, output-noise standard deviation, and analytic per-output failure-probability bounds.

## 7 Conclusion

We presented a BFV-based functional bootstrapping method for arbitrary functions over large plaintext spaces that uses non-amortized $\tilde{O}(1)$ polynomial multiplications for a single LWE ciphertext and extends naturally to small and moderate batches. Its main ingredient is a batched homomorphic polynomial-evaluation algorithm for sparsely packed BFV ciphertexts. When $d < (N/m)^2$ and $d = O(N)$, it evaluates $m$ polynomials on $m$ encrypted inputs using $\tilde{O}(m)$

polynomial multiplications, giving amortized $\tilde{O}(1)$ complexity per input. Combined with the framework of [47], this yields a large-plaintext functional bootstrapping scheme suited to workloads involving only a moderate number of LWE ciphertexts. Our Lattigo implementation supports plaintext spaces of 9, 12, 14, and 16 bits and achieves practical performance for both single-ciphertext and small-batch evaluation.

**Table 5.** Parameters used for the functional bootstrapping experiments

| Parameter set | $N$ | $\log(PQ)$ | $d$ | $q$ | $p$ | $\log(Q)$ | | | | $\log(P)$ |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | Packing | PolyEval | StC | Base | |
| 9-bit ($m = 1$) | 32768 | 868 | $2^{16}$ | 65537 | $2^9$ | 25 | $39 \times 18$ | 0 | 47 | $47 \times 2$ |
| 9-bit ($m = 128$) | 32768 | 868 | $2^{16}$ | 65537 | $2^9$ | 30 | $37 \times 18$ | 37 | 45 | $45 \times 2$ |
| 12-bit ($m = 1, 64$) | 65536 | 1660 | $2^{20} - 1$ | 786433 | $2^{12}$ | 30 | $52 \times 23$ | 52 | 52 | $55 \times 6$ |
| 14-bit ($m = 1, 32$) | 65536 | 1712 | $2^{22} - 1$ | 2752513 | $2^{14}$ | 30 | $52 \times 24$ | 52 | 52 | $55 \times 6$ |
| 16-bit ($m = 1$) | 65536 | 1717 | $2^{23} - 1$ | 8257537 | $2^{16}$ | 30 | $52 \times 25$ | 59 | 53 | $55 \times 5$ |
| 16-bit ($m = 16$) | 65536 | 1723 | $2^{23} - 1$ | 8257537 | $2^{16}$ | 30 | $59, 52 \times 24$ | 59 | 52 | $55 \times 5$ |

* The remaining Base modulus is the modulus level used for the final RLWE-to-LWE key switching.
† For $d = 2^{16}$, we first apply Algorithm 5 to the degree-$(d-1)$ part, for which $(d-1)+1 = 2^{16}$, and then add the remaining term $a_d x^d$.
‡ For the single-ciphertext rows with $m = 1$, the sparse SlotToCoeff transform is skipped. If a nonzero modulus budget is displayed under StC, that budget is used instead for an additional modulus switch after polynomial evaluation to control the accumulated noise.

**Table 6.** Runtime breakdown of functional bootstrapping (seconds)

| $m$ | $p$ | Bootstrap key (GiB) | Step 1 | Step 2 | | | Step 3 | Online total | Noise $\sigma_{\text{res}}$ | Per-LWE fail. prob. |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | EvalPower | BatchLT | Total | | | | |
| 1 | 512 | 1.68 | 1.365 | 1.625 | 0.110 | 1.753 | 0.032 | 3.149 | 10.1071 | $\leq 2^{-16.30}$ |
| 128 | 512 | 1.22 | 0.901 | 1.668 | 0.898 | 2.613 | 0.259 | 3.773 | 10.6473 | |
| 1 | 4096 | 2.68 | 2.967 | 4.773 | 0.455 | 5.327 | 0.065 | 8.359 | 10.6042 | $\leq 2^{-37.93}$ |
| 64 | 4096 | 2.25 | 1.925 | 4.698 | 6.694 | 11.587 | 0.428 | 13.940 | 10.6175 | |
| 1 | 16384 | 2.81 | 3.051 | 5.403 | 0.698 | 6.230 | 0.065 | 9.346 | 11.0545 | $\leq 2^{-28.80}$ |
| 32 | 16384 | 2.49 | 2.100 | 5.205 | 9.076 | 14.484 | 0.292 | 16.876 | 10.8440 | |
| 1 | 65536 | 3.41 | 3.396 | 6.054 | 0.984 | 7.167 | 0.066 | 10.629 | 11.2168 | $\leq 2^{-15.76}$ |
| 16 | 65536 | 3.21 | 2.551 | 5.877 | 9.276 | 15.340 | 0.183 | 18.073 | 12.1785 | |

[a] Times are averages over 100 independent runs and refer to the online bootstrapping phase.
[b] The bootstrap-key size includes the BFV encryptions of the LWE secret and all auxiliary keys. The noise column reports the empirical standard deviation of the output LWE noise.
[c] The last column reports the analytic failure-probability bound for each output LWE ciphertext.
[d] In the runtime breakdown in Table 6, we list the dominant parts separately. In particular, within Step 2, EvalPower and BatchLT are shown explicitly, since they account for most of the cost of homomorphic polynomial evaluation.
[e] The analytic failure bounds assume a dense secret of Hamming weight $h = 2048$; using a sparser secret, e.g., $h = 512$ as in [30], can further reduce the failure probability.

**Acknowledgements.** This research was supported by the National Key Research and Development Project under Grant No. 2025YFA1017200 and STU-Scientific Research Initiation Grant No. NTF24023T.

## References


1. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using CKKS. In: Advances in Cryptology – CRYPTO 2025, Part III. Lecture Notes in Computer Science, vol. 16002, pp. 304–337. Springer Nature Switzerland (2025). https://doi.org/10.1007/978-3-032-01881-6_10
2. Alperin-Sheriff, J., Peikert, C.: Practical bootstrapping in quasilinear time. In: Advances in Cryptology – CRYPTO 2013, Part I. Lecture Notes in Computer Science, vol. 8042, pp. 1–20. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_1
3. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with CKKS. In: Advances in Cryptology – ASIACRYPT 2024, Part I. Lecture Notes in Computer Science, vol. 15484, pp. 330–360. Springer Singapore (2024). https://doi.org/10.1007/978-981-96-0875-1_11
4. Bian, S., Zhang, Z., Pan, H., Mao, R., Zhao, Z., Jin, Y., Guan, Z.: $HE^3DB$: An efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2930–2944. ACM (2023). https://doi.org/10.1145/3576915.3616608
5. Boneh, D., Gentry, C., Halevi, S., Wang, F., Wu, D.J.: Private database queries using somewhat homomorphic encryption. In: Applied Cryptography and Network Security – ACNS 2013. Lecture Notes in Computer Science, vol. 7954, pp. 102–118. Springer (2013). https://doi.org/10.1007/978-3-642-38980-1_7
6. Bossuat, J.P., Cammarota, R., Chillotti, I., Curtis, B.R., Dai, W., Gong, H., Hales, E., Kim, D., Kumara, B., Lee, C., Lu, X., Maple, C., Pedrouzo-Ulloa, A., Player, R., Polyakov, Y., Ruiz Lopez, L.A., Song, Y., Yhee, D.: Security guidelines for implementing homomorphic encryption. IACR Communications in Cryptology **1**(4) (2025). https://doi.org/10.62056/anxra69p1
7. Bossuat, J.P., Mouchet, C., Troncoso-Pastoriza, J.R., Hubaux, J.P.: Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In: Advances in Cryptology – EUROCRYPT 2021, Part I. Lecture Notes in Computer Science, vol. 12696, pp. 587–617. Springer (2021). https://doi.org/10.1007/978-3-030-77870-5_21
8. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science, vol. 7417, pp. 868–886. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_50
9. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. pp. 309–325. ITCS '12, ACM (2012). https://doi.org/10.1145/2090236.2090262
10. Chen, H., Chillotti, I., Song, Y.: Improved bootstrapping for approximate homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2019, Part II. Lecture Notes in Computer Science, vol. 11477, pp. 34–54. Springer (2019). https://doi.org/10.1007/978-3-030-17656-3_2
11. Chen, H., Han, K.: Homomorphic lower digits removal and improved FHE bootstrapping. In: Advances in Cryptology – EUROCRYPT 2018, Part I. Lecture Notes in Computer Science, vol. 10820, pp. 315–337. Springer (2018). https://doi.org/10.1007/978-3-319-78381-9_12
12. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2018, Part

I. Lecture Notes in Computer Science, vol. 10820, pp. 360–384. Springer (2018). https://doi.org/10.1007/978-3-319-78381-9_14
13. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017, Part I. Lecture Notes in Computer Science, vol. 10624, pp. 409–437. Springer (2017). https://doi.org/10.1007/978-3-319-70694-8_15
14. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Advances in Cryptology – ASIACRYPT 2016, Part I. Lecture Notes in Computer Science, vol. 10031, pp. 3–33. Springer (2016). https://doi.org/10.1007/978-3-662-53887-6_1
15. Chillotti, I., Joye, M., Paillier, P.: Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In: Cyber Security Cryptography and Machine Learning – CSCML 2021. Lecture Notes in Computer Science, vol. 12716, pp. 1–19. Springer (2021). https://doi.org/10.1007/978-3-030-78086-9_1
16. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: Advances in Cryptology – ASIACRYPT 2021, Part III. Lecture Notes in Computer Science, vol. 13092, pp. 670–699. Springer (2021). https://doi.org/10.1007/978-3-030-92078-4_23
17. De Micheli, G., Kim, D., Micciancio, D., Suhl, A.: Faster amortized FHEW bootstrapping using ring automorphisms. In: Public-Key Cryptography – PKC 2024, Part IV. Lecture Notes in Computer Science, vol. 14604, pp. 322–353. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-57728-4_11
18. Duan, K., Li, H., Liu, D., Ma, G.: Large-plaintext functional bootstrapping in FHE with small bootstrapping keys. In: Theory of Cryptography – TCC 2025, Part I. Lecture Notes in Computer Science, vol. 16268, pp. 253–279. Springer Nature Switzerland (2026). https://doi.org/10.1007/978-3-032-12287-2_9
19. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Advances in Cryptology – EUROCRYPT 2015, Part I. Lecture Notes in Computer Science, vol. 9056, pp. 617–640. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_24
20. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. IACR Cryptology ePrint Archive p. 144 (2012), https://eprint.iacr.org/2012/144
21. Geelen, R., Iliashenko, I., Kang, J., Vercauteren, F.: On polynomial functions modulo $p^e$ and faster bootstrapping for homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2023, Part III. Lecture Notes in Computer Science, vol. 14006, pp. 257–286. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-30620-4_9
22. Geelen, R., Vercauteren, F.: Bootstrapping for BGV and BFV revisited. Journal of Cryptology **36**(2), 12 (2023). https://doi.org/10.1007/s00145-023-09454-6
23. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University (2009), https://crypto.stanford.edu/craig/craig-thesis.pdf
24. Gentry, C., Halevi, S., Smart, N.P.: Better bootstrapping in fully homomorphic encryption. In: Public-Key Cryptography – PKC 2012. Lecture Notes in Computer Science, vol. 7293, pp. 1–16. Springer (2012). https://doi.org/10.1007/978-3-642-30057-8_1, https://eprint.iacr.org/2011/680
25. Gentry, C., Halevi, S., Smart, N.P.: Fully homomorphic encryption with polylog overhead. In: Advances in Cryptology – EUROCRYPT 2012. Lecture Notes in Computer Science, vol. 7237, pp. 465–482. Springer (2012). https://doi.org/10.1007/978-3-642-29011-4_28

26. Gentry, C., Halevi, S., Smart, N.P.: Homomorphic evaluation of the AES circuit. In: Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science, vol. 7417, pp. 850–867. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_49
27. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Advances in Cryptology – CRYPTO 2013, Part I. Lecture Notes in Computer Science, vol. 8042, pp. 75–92. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_5
28. Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., Wernsing, J.: CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In: Proceedings of the 33rd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 48, pp. 201–210. PMLR (2016), https://proceedings.mlr.press/v48/gilad-bachrach16.html
29. Guimarães, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in TFHE. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 229–253 (2021). https://doi.org/10.46586/tches.v2021.i2.229-253
30. Guimarães, A., Pereira, H.V.L.: Fast amortized bootstrapping with small keys and polynomial noise overhead. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). pp. 2967–2981. Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3719027.3765181
31. Guimarães, A., Pereira, H.V.L., van Leeuwen, B.: Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In: Advances in Cryptology – ASIACRYPT 2023, Part VI. Lecture Notes in Computer Science, vol. 14443, pp. 3–35. Springer Nature Singapore (2023). https://doi.org/10.1007/978-981-99-8736-8_1
32. Halevi, S., Shoup, V.: Algorithms in HElib. In: Garay, J.A., Gennaro, R. (eds.) Advances in Cryptology – CRYPTO 2014, Part I, Lecture Notes in Computer Science, vol. 8616, pp. 554–571. Springer, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44371-2_31
33. Halevi, S., Shoup, V.: Bootstrapping for HElib. In: Advances in Cryptology – EUROCRYPT 2015, Part I. Lecture Notes in Computer Science, vol. 9056, pp. 641–670. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_25
34. Hong, S., Park, J.H., Cho, W., Choe, H., Cheon, J.H.: Secure tumor classification by shallow neural network using homomorphic encryption. BMC Genomics **23**(1), 284 (2022). https://doi.org/10.1186/s12864-022-08469-w
35. Juvekar, C., Vaikuntanathan, V., Chandrakasan, A.: GAZELLE: A low latency framework for secure neural network inference. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 1651–1669. USENIX Association (2018), https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar
36. Kim, A., Polyakov, Y., Zucca, V.: Revisiting homomorphic encryption schemes for finite fields. In: Advances in Cryptology – ASIACRYPT 2021, Part III. Lecture Notes in Computer Science, vol. 13092, pp. 608–639. Springer (2021). https://doi.org/10.1007/978-3-030-92078-4_21
37. Kim, D., Song, Y.: Approximate homomorphic encryption over the conjugate-invariant ring. In: Lee, K. (ed.) Information Security and Cryptology – ICISC 2018, Lecture Notes in Computer Science, vol. 11396, pp. 85–102. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-12146-4_6
38. Kluczniak, K., Schild, L.: FDFB: Full-domain functional bootstrapping towards practical fully homomorphic encryption. IACR Transactions on Cryptographic

Hardware and Embedded Systems **2023**(1), 501–537 (2023). https://doi.org/10.46586/tches.v2023.i1.501-537
39. Lattigo v6.2.0. Online software release: https://github.com/tuneinsight/lattigo/releases/tag/v6.2.0 (Feb 2026), EPFL-LDS, Tune Insight SA; released 2 February 2026
40. Lee, D., Min, S., Song, Y.: Functional bootstrapping for packed ciphertexts via homomorphic LUT evaluation. Cryptology ePrint Archive, Paper 2024/181 (2024), https://eprint.iacr.org/2024/181
41. Lee, Y., Lee, J.W., Kim, Y.S., Kim, Y., No, J.S., Kang, H.: High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In: Advances in Cryptology – EUROCRYPT 2022, Part I. Lecture Notes in Computer Science, vol. 13275, pp. 551–580. Springer (2022). https://doi.org/10.1007/978-3-031-06944-4_19
42. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2023, Part III. Lecture Notes in Computer Science, vol. 14006, pp. 227–256. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-30620-4_8
43. Li, Z., Wei, B., Wang, R., Lu, X., Wang, K.: Full domain functional bootstrapping with least significant bit encoding. In: Information Security and Cryptology – Inscrypt 2023, Part I. Lecture Notes in Computer Science, vol. 14526, pp. 203–223. Springer Nature Singapore (2024). https://doi.org/10.1007/978-981-97-0942-7_11
44. Liu, F.H., Wang, H.: Batch bootstrapping I: A new framework for SIMD bootstrapping in polynomial modulus. In: Advances in Cryptology – EUROCRYPT 2023, Part III. Lecture Notes in Computer Science, vol. 14006, pp. 321–352. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-30620-4_11
45. Liu, F.H., Wang, H.: Batch bootstrapping II: Bootstrapping in polynomial modulus only requires $\widetilde{O}(1)$ FHE multiplications in amortization. In: Advances in Cryptology – EUROCRYPT 2023, Part III. Lecture Notes in Computer Science, vol. 14006, pp. 353–384. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-30620-4_12
46. Liu, Z., Micciancio, D., Polyakov, Y.: Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. In: Advances in Cryptology – ASIACRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13792, pp. 130–160. Springer Nature Switzerland (2022). https://doi.org/10.1007/978-3-031-22966-4_5
47. Liu, Z., Wang, Y.: Amortized functional bootstrapping in less than 7 ms, with $\widetilde{O}(1)$ polynomial multiplications. In: Advances in Cryptology – ASIACRYPT 2023, Part VI. Lecture Notes in Computer Science, vol. 14443, pp. 101–132. Springer Nature Singapore (2023). https://doi.org/10.1007/978-981-99-8736-8_4
48. Lu, W.j., Huang, Z., Hong, C., Ma, Y., Qu, H.: PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1057–1073. IEEE (2021). https://doi.org/10.1109/SP40001.2021.00043
49. Ma, S., Huang, T., Wang, A., Zhou, Q., Wang, X.: Fast and accurate: Efficient full-domain functional bootstrap and digit decomposition for homomorphic computation. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 592–616 (2024). https://doi.org/10.46586/tches.v2024.i1.592-616
50. Micciancio, D., Sorrell, J.: Ring packing and amortized FHEW bootstrapping. In: 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018). Leibniz International Proceedings in Informatics (LIPIcs), vol. 107,

pp. 100:1–100:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2018). https://doi.org/10.4230/LIPIcs.ICALP.2018.100
51. Okada, H., Player, R., Pohmann, S.: Homomorphic polynomial evaluation using Galois structure and applications to BFV bootstrapping. In: Advances in Cryptology – ASIACRYPT 2023, Part VI. Lecture Notes in Computer Science, vol. 14443, pp. 69–100. Springer (2023). https://doi.org/10.1007/978-981-99-8736-8_3
52. Park, J., Park, S., Park, J.H., Ahn, J.H., Cheon, J.H., Hanrot, G., Kim, J.W., Park, M., Stehlé, D.: Scaling up privacy-preserving ML: A CKKS implementation of Llama-2-7B. arXiv preprint arXiv:2601.18511v1 (2026). https://doi.org/10.48550/arXiv.2601.18511, https://arxiv.org/abs/2601.18511v1, version 1, submitted 26 January 2026
53. Paterson, M.S., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. SIAM Journal on Computing **2**(1), 60–66 (1973). https://doi.org/10.1137/0202007
54. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Proceedings of the 37th Annual ACM Symposium on Theory of Computing. pp. 84–93. ACM (2005). https://doi.org/10.1145/1060590.1060603
55. Wang, Q., Wang, L.P.: A novel asymmetric BSGS polynomial evaluation algorithm under homomorphic encryption. In: Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. pp. 30–44. ACM (2025). https://doi.org/10.1145/3708821.3710822
56. Wang, R., Lu, X., Wen, Y., Li, Z., Wei, B., Wang, K., Luo, L.: Full domain functional bootstrapping using the prime cyclotomic ring. Theoretical Computer Science **1050**, 115392 (2025). https://doi.org/10.1016/j.tcs.2025.115392
57. Xia, H., Liu, F.H., Wang, H.: More efficient functional bootstrapping for general functions in polynomial modulus. In: Theory of Cryptography – TCC 2024, Part IV. Lecture Notes in Computer Science, vol. 15367, pp. 130–163. Springer Nature Switzerland (2025). https://doi.org/10.1007/978-3-031-78023-3_5
58. Xu, Y., Zhu, G., Wang, H., Wang, L.P.: Large-plaintext functional bootstrapping with small parameters for BGV encoding. In: Information Security – ISC 2024, Part I. Lecture Notes in Computer Science, vol. 15257, pp. 252–272. Springer Nature Switzerland (2025). https://doi.org/10.1007/978-3-031-75757-0_13
59. Yalçın, E., Tanyıldız, H., Aslan, S., Demir, S.C., Sucu, M., Uzay, F.İ., Aykut, S., Yalçın, E.M., Biçer, A.: Explainable tabular deep learning models for antenatal cesarean delivery prediction in multiparous women. BMC Pregnancy and Childbirth **26**(1), 453 (2026). https://doi.org/10.1186/s12884-026-08934-4
60. Yu, Y., Zhou, Y., Chen, Y., Soto, P., Xiong, W., Li, M.: Cachemir: Fully homomorphic encrypted inference of generative large language model with KV cache. arXiv preprint arXiv:2602.11470 (2026). https://doi.org/10.48550/arXiv.2602.11470, https://arxiv.org/abs/2602.11470
61. Zhao, J., Zhang, J., Xiang, B., Wu, S., Deng, Y., Feng, D.: Faster polynomial evaluations for SIMD FHEs and application to BGV in HElib. In: Heninger, N., Rosulek, M. (eds.) Advances in Cryptology – CRYPTO 2026, Part II. Lecture Notes in Computer Science, vol. 16801, pp. 461–491. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-35374-0_15